Article-1

Task 1: Prevent Two Pipeline Jobs of the Same Type from Running in Parallel on the Same Node

1. Objective

To ensure that no two instances of the same Jenkins pipeline job execute simultaneously on the same node, thereby preventing resource contention, workspace conflicts, and data corruption.

2. Purpose

When two builds of the same job run in parallel on one node, they compete for CPU, memory, and disk I/O. They also share the same workspace directory, which can cause files to be overwritten, corrupted, or produce unpredictable build outcomes. This task solves that problem by enforcing exclusive execution.

3. Prerequisites

Item	Requirement
Jenkins Version	2.580.1 or later
Agents	jenkins-agent-01 (label: agent-01), jenkins-agent-02 (label: agent-02)
Job Type	Pipeline (Declarative)
Access	Admin or job-configure permission

4. Plugins Needed

Method	Plugin Required
Method 1: disableConcurrentBuilds()	None — built into Jenkins core
Method 2: Throttle Concurrent Builds	**Throttle Concurrent Builds Plugin**
Method 3: Lockable Resources	**Lockable Resources Plugin**

5. Methods and Step-by-Step Procedures

Method 1: disableConcurrentBuilds() — Built-in

Step 1: Click **New Item** → Name: Concurrency-Demo → Select **Pipeline** → **OK**

Step 2: In the Pipeline script box, paste:

```
pipeline {
  agent any
  options {
    disableConcurrentBuilds()
  }
  stages {
    stage('Build') {
      steps {
        echo "Running on: ${env.NODE_NAME}"
        sleep 60
        echo 'Build done.'
      }
    }
  }
}
```

Step 3: Click **Save** → Click **Build Now** twice rapidly.
Step 4: Observe the Build Queue — the second build waits for the first to complete.
Result: Only one build of this job runs at a time across the entire Jenkins instance.

Method 2: Throttle Concurrent Builds Plugin — Per-Node Control
Step 1: Go to **Manage Jenkins → Plugins → Available** → Search Throttle Concurrent Builds → Install without restart.
Step 2: Go to **Manage Jenkins → System** → Scroll to **Throttle Concurrent Builds** → Add a category:

- Category Name: MyThrottleCategory
- Maximum Concurrent Builds Per Node: 1
- Maximum Total Concurrent Builds: 2

Step 3: Save. Update the pipeline:

```
pipeline {
  agent { label 'agent-01' }
  options {
    throttleJobProperty(
      categories: ['MyThrottleCategory'],
      throttleEnabled: true,
      throttleOption: 'category'
    )
  }
  stages {
    stage('Build') {
      steps {
        echo "Running on: ${env.NODE_NAME}"
        sleep 60
      }
    }
  }
}
```

Step 4: Save → Build Now twice.
Result: Only one build from the category runs per node. Parallel builds across different nodes are still allowed.

Method 3: Lockable Resources Plugin — Fine-Grained Locking

Step 1: Install **Lockable Resources Plugin** via **Manage Jenkins → Plugins → Available**.
Step 2: Go to **Manage Jenkins → System → Lockable Resources** → Add a resource:

- Name: MyJobResource
- Description: Resource for concurrency demo

Step 3: Save. Update the pipeline:

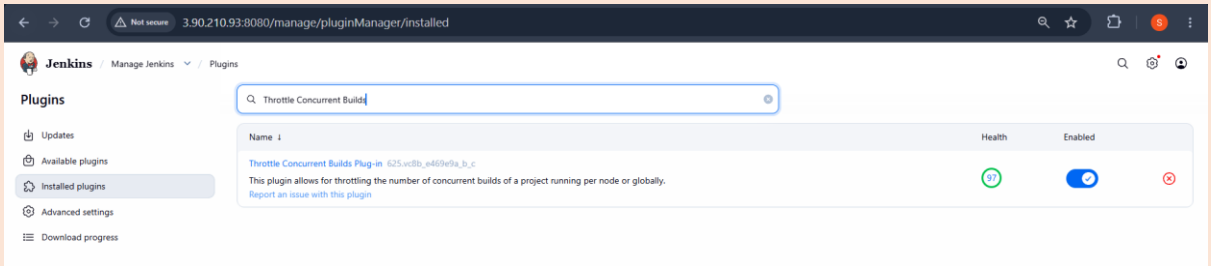
```
pipeline {
   agent any
   stages {
      stage('Preparation') {
         steps {
            lock(resource: 'MyJobResource') {
               echo 'Lock acquired, running the critical section'
               sleep 30
            }
         }
      }
   }
}
```

Step 4: Save → Build Now twice.
Result: Only one build acquires the lock at a time. Other builds wait.

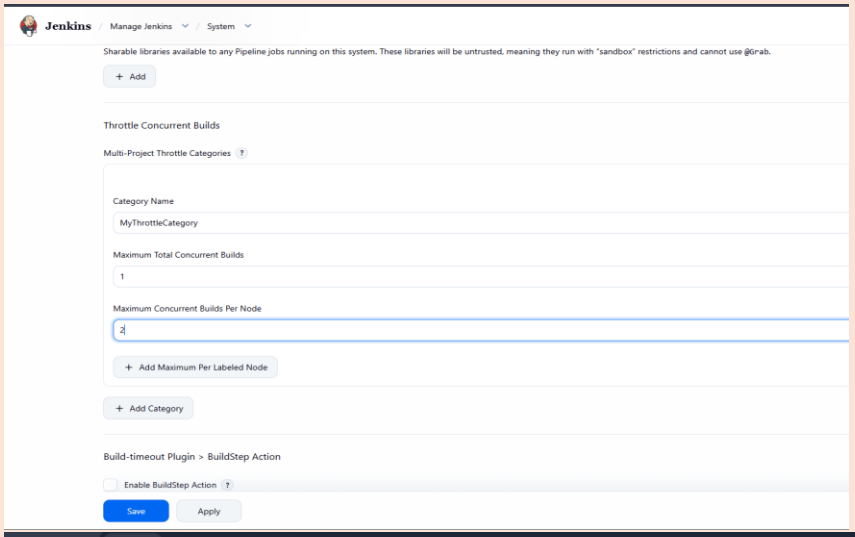
6. Verification

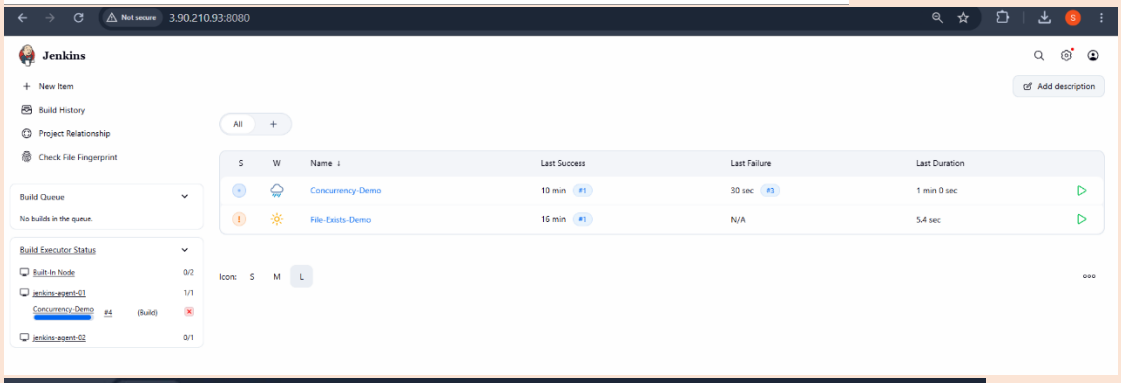
- Go to **Build Queue** — waiting build shows: *"Build #N is waiting for the completion of Build #M"*
- Go to **Build Executor Status** — only one executor busy at a time
- Go to **Manage Jenkins → System → Lockable Resources** — resource shows locked/free status

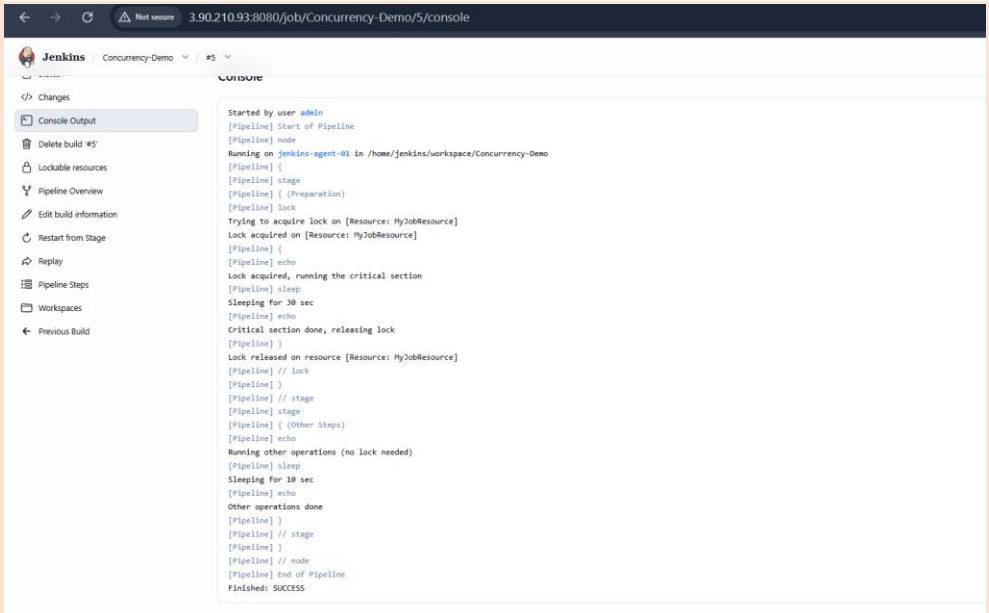
Implementation:

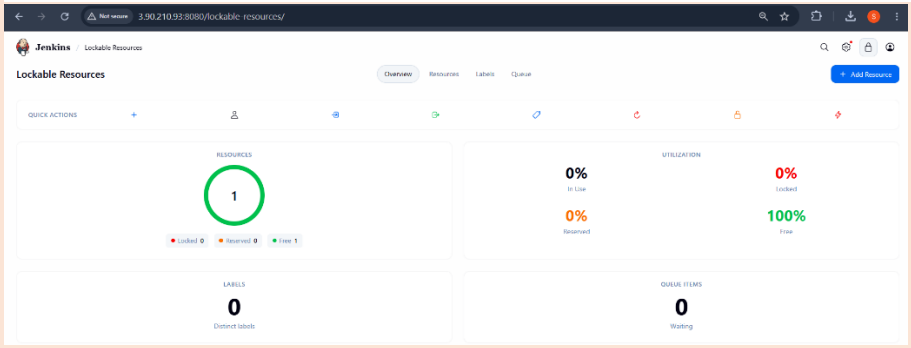


Article-1


Jenkins
Manage Jenkins
System
Sharable libraries available to any Pipeline jobs running on this system. These libraries will be untrusted, meaning they run with "sandbox" restrictions and cannot use @Grab.
+ Add
Throttle Concurrent Builds
Multi-Project Throttle Categories
Category Name
MyThrottleCategory
Maximum Total Concurrent Builds
1
Maximum Concurrent Builds Per Node
2
+ Add Maximum Per Labeled Node
+ Add Category
Build-timeout Plugin > BuildStep Action
Enable BuildStep Action
Save
Apply


Not secure
3.90.210.93:8080
Jenkins
New Item
Build History
Project Relationship
Check File Fingerprint
Add description
All
S
W
Name
Last Success
Last Failure
Last Duration
Concurrency-Demo
10 min #1
30 sec #3
1 min 0 sec
File-Exists-Demo
16 min #1
N/A
5.4 sec
Build Queue
No builds in the queue.
Build Executor Status
Built-In Node
0/2
jenkins-agent-01
1/1
Concurrency-Demo #4 (Build)
jenkins-agent-02
0/1
Icon: S M L


Not secure
3.90.210.93:8080/job/Concurrency-Demo/5/console
Jenkins
Concurrency-Demo
#5
Changes
Console Output
Delete build '#5'
Lockable resources
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/Concurrency-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Preparation)
[Pipeline] lock
Trying to acquire lock on [Resource: MyJobResource]
Lock acquired on [Resource: MyJobResource]
[Pipeline] {
[Pipeline] echo
Lock acquired, running the critical section
[Pipeline] sleep
Sleeping for 30 sec
[Pipeline] echo
Critical section done, releasing lock
[Pipeline] }
Lock released on resource [Resource: MyJobResource]
[Pipeline] // lock
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Other Steps)
[Pipeline] echo
Running other operations (no lock needed)
[Pipeline] sleep
Sleeping for 10 sec
[Pipeline] echo
Other operations done
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Not secure
3.90.210.93:8080/lockable-resources/
Jenkins
Lockable Resources
Lockable Resources
Overview
Resources
Labels
Queue
Add Resource
QUICK ACTIONS
RESOURCES
1
Locked 0
Reserved 0
Free 1
UTILIZATION
0%
In Use
0%
Locked
0%
Reserved
100%
Free
LABELS
0
Distinct labels
QUEUE ITEMS
0
Waiting

7. Conclusion
All three methods successfully prevent parallel execution on the same node. disableConcurrentBuilds() is simplest and global. The Throttle plugin allows cross-node parallelism with per-node limits. Lockable Resources provides fine-grained locking for specific critical sections. Choose based on whether you need global strictness, per-node limits, or selective locking.

Task 2: Check if a File Exists in a Jenkins Pipeline

1. Objective

To verify the existence of a file within a Jenkins pipeline workspace before proceeding with subsequent stages such as archiving, deployment, or conditional logic.

2. Purpose

Pipelines often depend on artifacts (e.g., build.jar, deploy-package.zip) or configuration files. Attempting to archive, deploy, or process a missing file causes failures and wasted build time. This task demonstrates how to check file existence safely and take appropriate action.

3. Prerequisites

Item	Requirement
Jenkins Version	2.580.1 or later
Agents	agent-01, agent-02 (both online)
Job Type	Pipeline (Declarative)
Access	Admin or job-configure permission

4. Plugins Needed

Method	Plugin Required
fileExists step	None — built into Jenkins Pipeline
Shell check (sh)	None — requires Linux/macOS agent
Batch check (bat)	None — requires Windows agent
Artifact archiving	None — built-in archiveArtifacts
Shared Library function	Jenkins Shared Library configured

5. Methods and Step-by-Step Procedures

Method 1: fileExists — Built-in Step

Step 1: Create job File-Exists-Demo-2 (Pipeline).

Step 2: Paste:

```
pipeline {
   agent { label 'agent-01' }
```

```
    stages {
        stage('Create File') {
            steps {
                sh 'echo "Hello from Jenkins" > myfile.txt'
            }
        }
        stage('Check File with fileExists') {
            steps {
                script {
                    if (fileExists('myfile.txt')) {
                        echo "✅ File found: myfile.txt"
                    } else {
                        echo "❌ File not found: myfile.txt"

                    }

                }

            }

        }

        stage('Check Non-Existent File') {
            steps {
                script {
                    if (fileExists('missing-file.txt')) {
                        echo "✅ File found: missing-file.txt"
                    } else {
                        echo "❌ File not found: missing-file.txt"
                    }
                }
            }
        }
    }
}
```

Step 3: Save → Build Now.

Result: myfile.txt found; missing-file.txt not found.

Method 2: Shell Command Check

```
stage('Check with Shell') {
    steps {
        script {
            def result = sh(
                script: '[ -f myfile.txt ] && echo "YES" || echo "NO"',
                returnStdout: true
            ).trim()
            if (result == "YES") {
                echo "✅ File exists (shell check)."
            } else {
```

```
            echo "❌ File missing (shell check)."
        }
    }
  }
}
```

Method 3: script { } Wrapper in Declarative Pipeline
Rule: Raw if statements cannot appear directly in Declarative steps { }. They must be wrapped in script { }.

```
stage('Check File') {
  steps {
    script {
      if (fileExists('myfile.txt')) {
        echo "Found!"
      }
    }
  }
}
```

Method 4: Handling the Outcome
Option A — Abort on critical file missing

```
script {
  if (!fileExists('deploy-package.zip')) {
    error "Package not found. Aborting."
  }
}
```

Option B — Mark UNSTABLE but continue:

```
script {
  if (!fileExists('optional-report.txt')) {
    currentBuild.result = 'UNSTABLE'
    echo "Report missing, marking unstable."
  }
}
```

Option C — Archive if present:

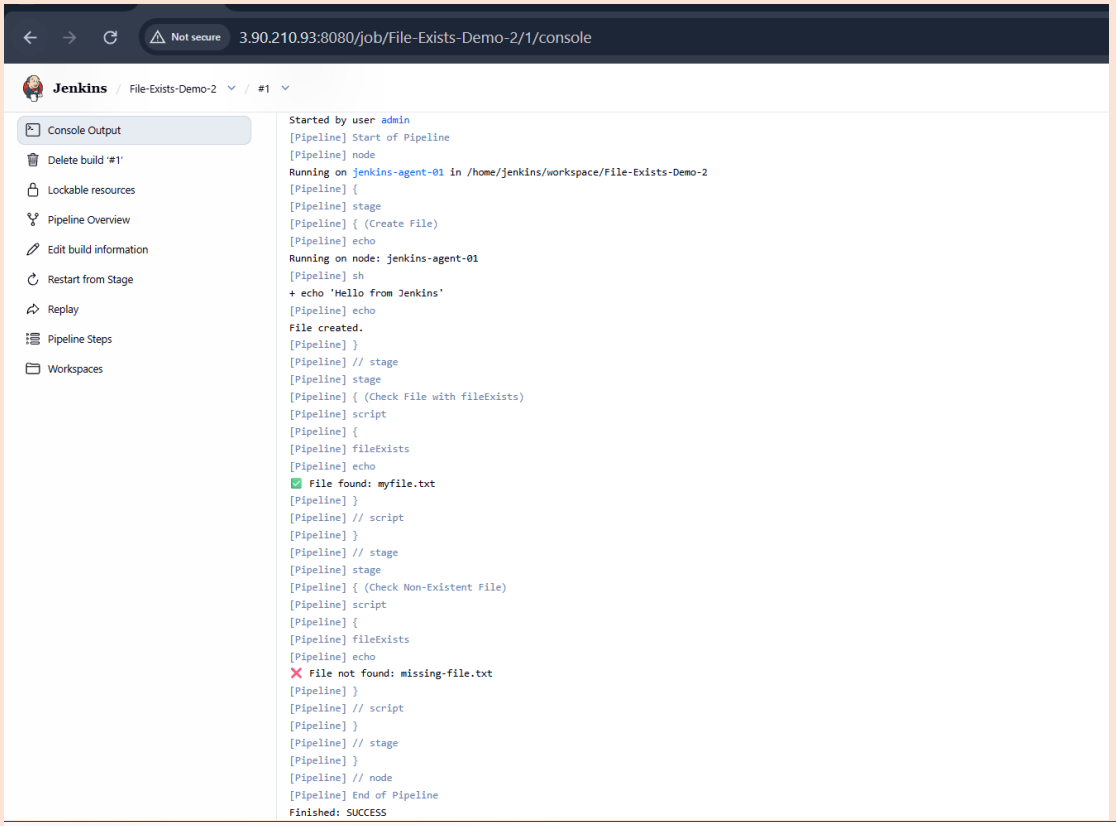
```
script {
  if (fileExists('build-output.jar')) {
    archiveArtifacts artifacts: 'build-output.jar', fingerprint: true
  }
}
```

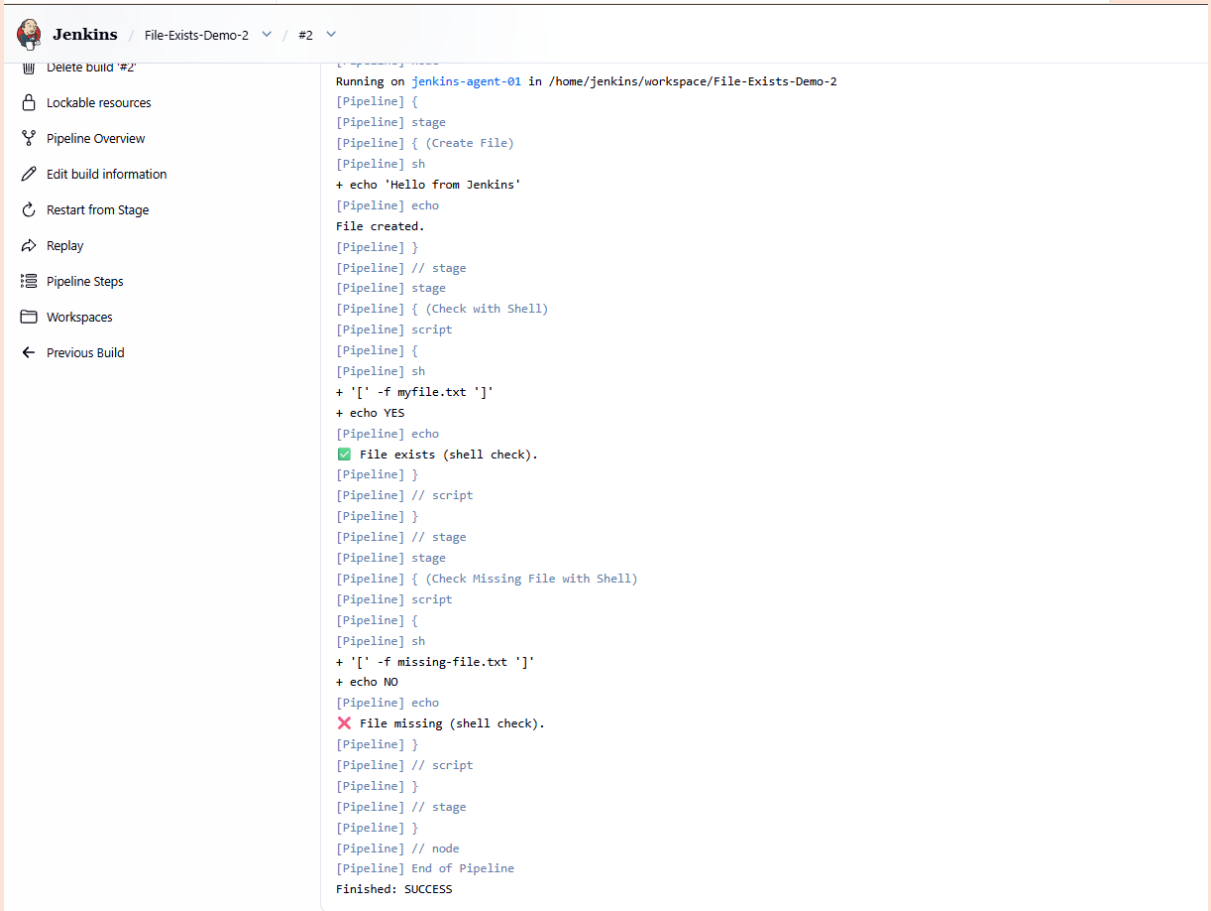
6. Verification

- Console output shows ✅ or ❌ based on file existence
- Build #4 (outcome handling) archived build-output.jar — visible in Artifacts section of build page
- Build #5 (failure path) aborted with Finished: FAILURE
- agent-02 build shows ❌ — proving workspace isolation

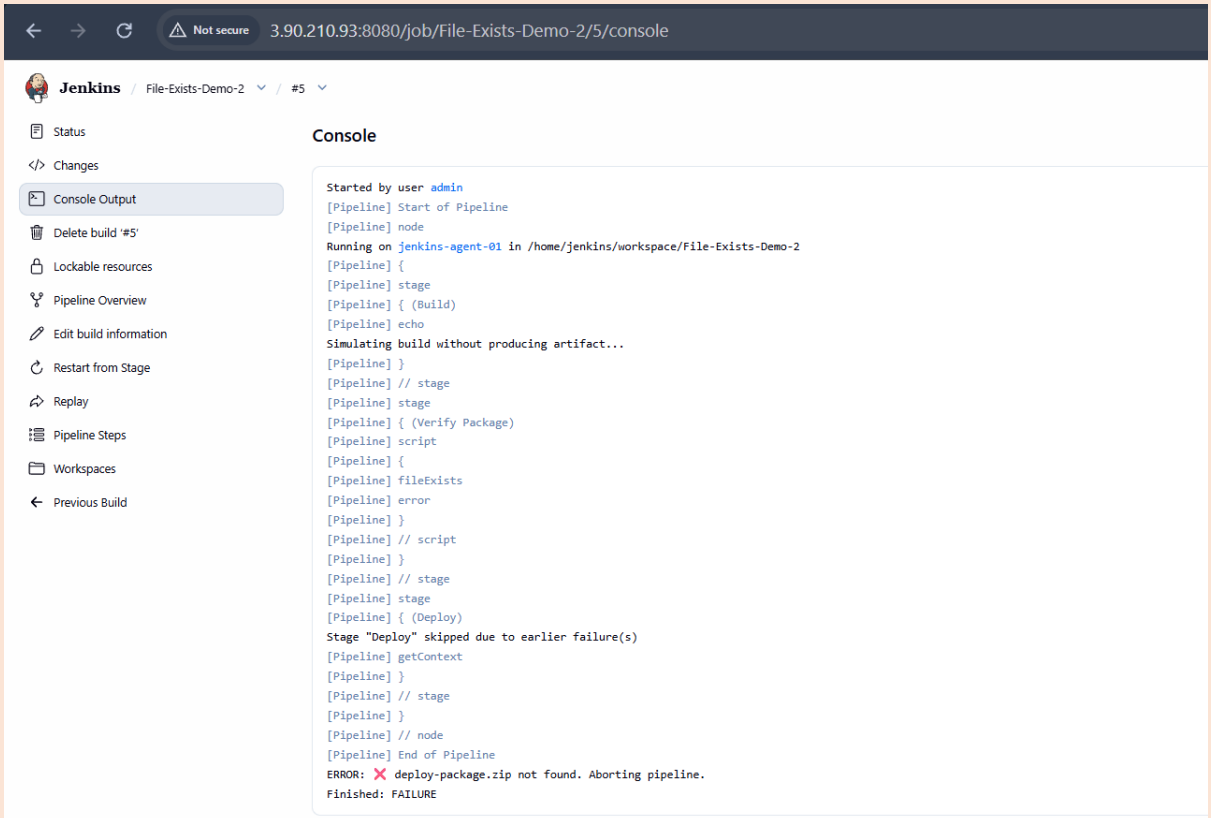
Article-1

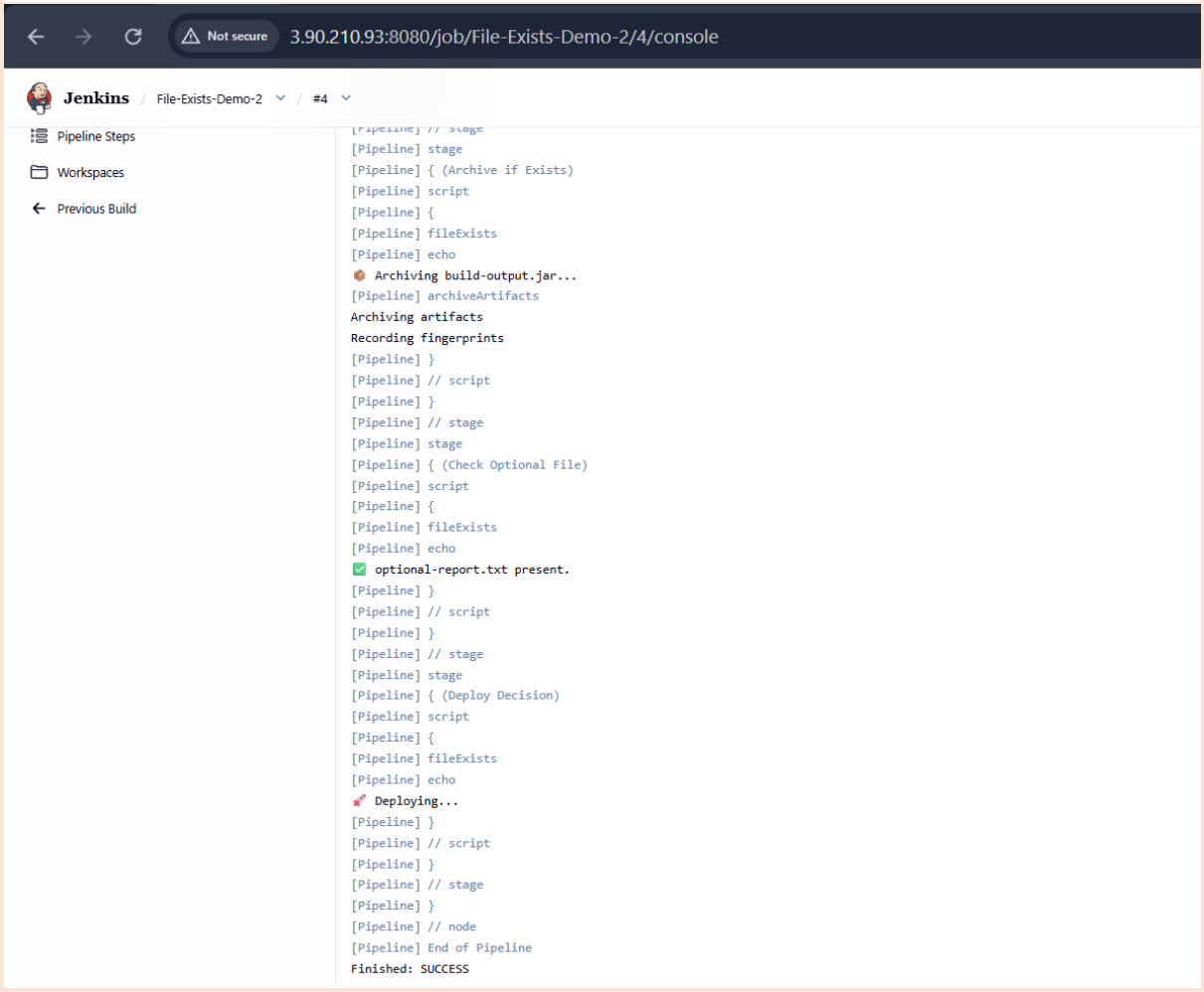
Impementation

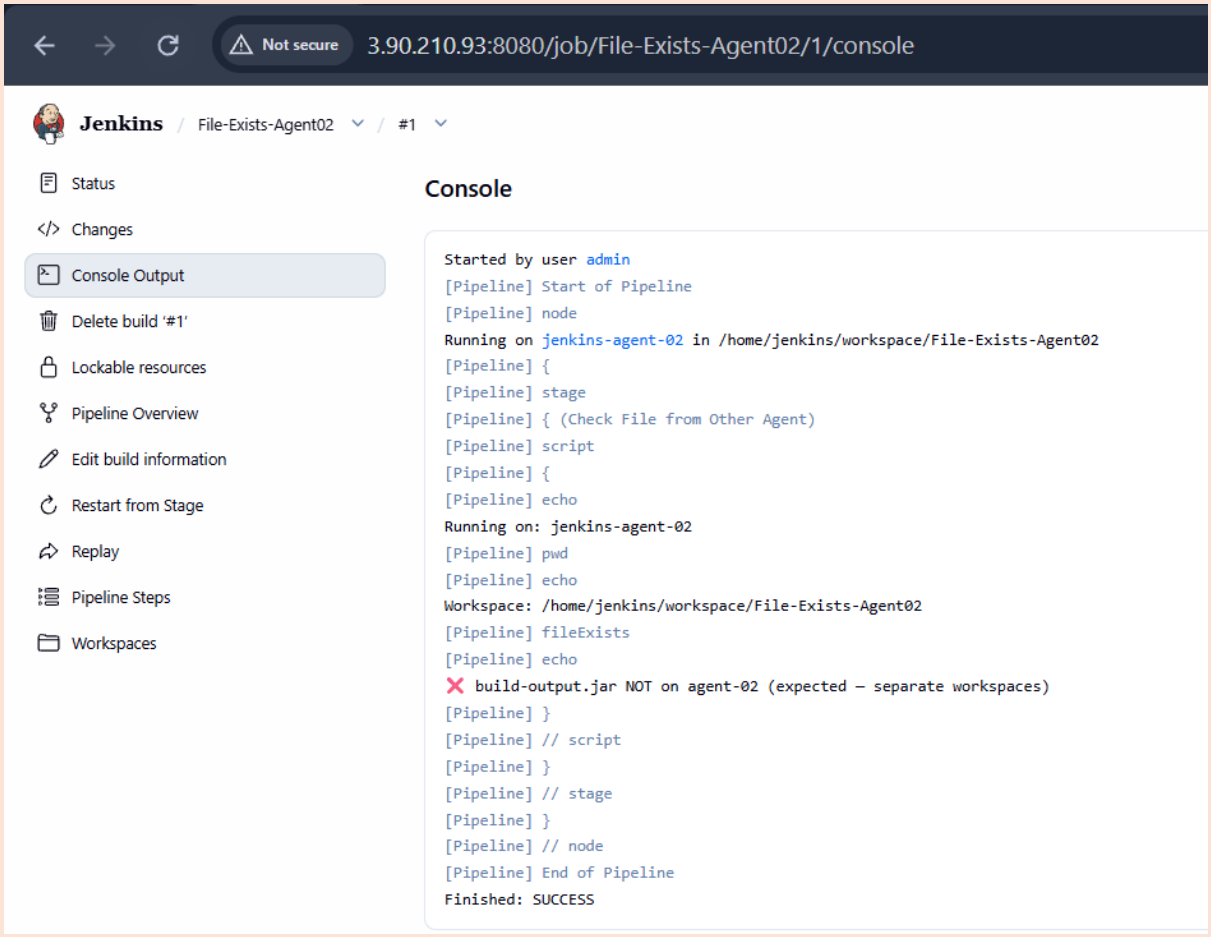




Article-1


Not secure
3.90.210.93:8080/job/File-Exists-Demo-2/5/console
Jenkins
File-Exists-Demo-2
#5
Status
Changes
Console Output
Delete build '#5'
Lockable resources
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
Console
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/File-Exists-Demo-2
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Simulating build without producing artifact...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Package)
[Pipeline] script
[Pipeline] {
[Pipeline] fileExists
[Pipeline] error
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
Stage "Deploy" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: ❌ deploy-package.zip not found. Aborting pipeline.
Finished: FAILURE


Not secure
3.90.210.93:8080/job/File-Exists-Demo-2/4/console
Jenkins
File-Exists-Demo-2
#4
Pipeline Steps
Workspaces
Previous Build
[Pipeline] stage
[Pipeline] { (Archive if Exists)
[Pipeline] script
[Pipeline] {
[Pipeline] fileExists
[Pipeline] echo
📦 Archiving build-output.jar...
[Pipeline] archiveArtifacts
Archiving artifacts
Recording fingerprints
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Check Optional File)
[Pipeline] script
[Pipeline] {
[Pipeline] fileExists
[Pipeline] echo
✅ optional-report.txt present.
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy Decision)
[Pipeline] script
[Pipeline] {
[Pipeline] fileExists
[Pipeline] echo
🚀 Deploying...
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS



7. Conclusion

The fileExists step is the simplest and most reliable method for checking files in the local workspace. Shell checks are useful when you prefer system commands. All conditional logic in Declarative pipelines must be wrapped in script { }. The error step aborts the pipeline for critical files; currentBuild.result = 'UNSTABLE' flags non-critical issues; archiveArtifacts conditionally stores build outputs. Crucially, fileExists only checks the local workspace of the current node — remote files require SSH or copy-first strategies.

Task 3: Set Stage Status in Jenkins Pipelines

1. Objective

To manually control the status (SUCCESS, FAILED, UNSTABLE) of individual pipeline stages regardless of the actual execution result of the steps inside them.

2. Purpose

Not every failure should stop the entire pipeline. Some steps — such as linting, reporting, or optional checks — may fail without being critical. This task demonstrates how to:

- Continue the pipeline despite a failure while flagging it (UNSTABLE)
- Force a SUCCESS status even when a command fails
- Understand the automatic skip behavior when a stage fails

Article-1

3. Prerequisites

Item	Requirement
Jenkins Version	2.580.1 or later
Agents	agent-01 (online)
Job Type	Pipeline (Declarative)
Access	Admin or job-configure permission

4. Plugins Needed

Step	Plugin Required
warnError	None — built into Jenkins Pipeline
catchError	None — built into Jenkins Pipeline
Stage View	None — built-in UI feature

5. Stage Status Reference

Status	Color	Pipeline Continues?	Meaning
SUCCESS	🟢 Green	✅ Yes	All steps passed
FAILED	🔴 Red	❌ No	Critical error; pipeline stops
UNSTABLE	🟡 Yellow	✅ Yes	Non-critical error; pipeline continues

Jenkins → Manage System → plugins → Available plugins → Pipeline: Stage View.

6. Methods and Step-by-Step Procedures

Part 1: warnError — Automatic UNSTABLE on Failure

Step 1: Create job Stage-Status-Demo (Pipeline).

Step 2: Paste:

```
pipeline {
   agent { label 'agent-01' }
   stages {
      stage('Success Stage') {
         steps {
            sh "exit 0"
            sh "ls -l"
         }
      }
      stage('Unstable Stage') {
         steps {
            warnError("The command failed") {
               sh "Wrong command"
            }
         }
      }
      stage('Continue Execution') {
         steps {
            sh "exit 0"
```

```
        }
      }
   }
}
```

Step 3: Save → Build Now.
Result: Stage 1 green, Stage 2 yellow, Stage 3 green. Build result: UNSTABLE. Pipeline continues.

Part 2: Default FAILED Behavior (No Wrapper)

```
pipeline {
   agent { label 'agent-01' }
   stages {
      stage('Success Stage') {
         steps { sh "exit 0" }
      }
      stage('Failed Stage') {
         steps {
            sh "exit 1"
         }
      }
      stage('Never Runs') {
         steps { echo "You should NOT see this." }
      }
   }
}
```

Result: Stage 1 green, Stage 2 red, Stage 3 skipped. Build result: FAILURE. Pipeline stops.
Part 3: catchError — Force SUCCESS

```
pipeline {
   agent { label 'agent-01' }
   stages {
      stage('Success Stage') {
         steps { sh "exit 0" }
      }
      stage('Set Stage Status to SUCCESS') {
         steps {
            catchError(buildResult: 'SUCCESS', stageResult: 'SUCCESS') {
               sh "exit 1"
            }
         }
      }
      stage('Continue Execution') {
         steps { sh "exit 0" }
      }
   }
}
```

Result: All stages green. Build result: SUCCESS. Failure ignored.

Part 4: catchError — Force UNSTABLE

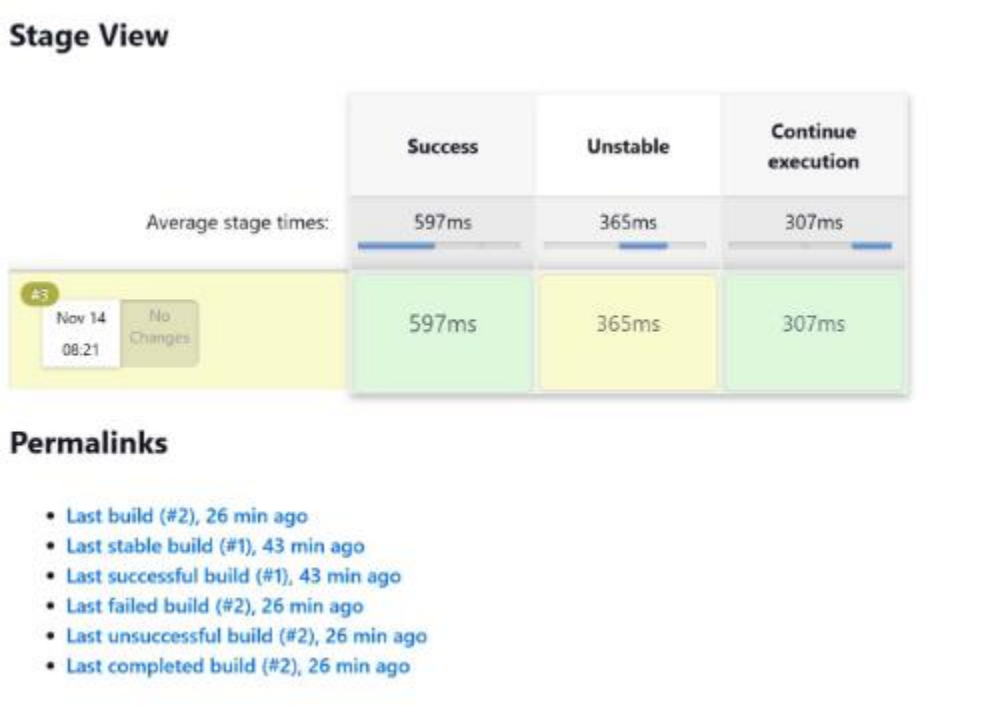
```
pipeline {
   agent { label 'agent-01' }
   stages {
      stage('Success Stage') {
         steps { sh "exit 0" }
      }
      stage('Set Stage Status to UNSTABLE') {
         steps {
            catchError(buildResult: 'UNSTABLE', stageResult: 'UNSTABLE') {
               sh "exit 1"
            }
         }
      }
      stage('Continue Execution') {
         steps { sh "exit 0" }
      }
   }
}
```

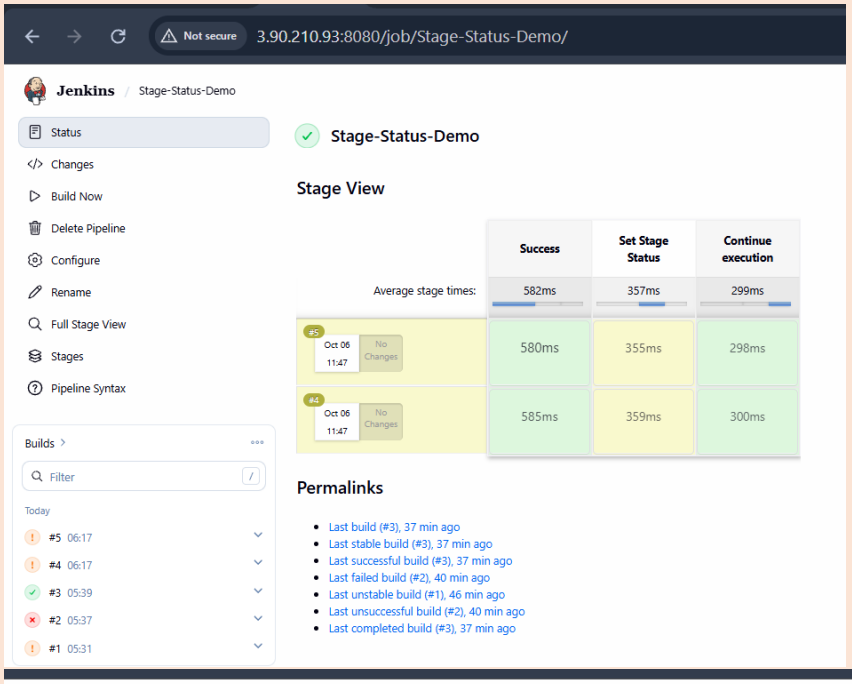
Result: Stage 1 green, Stage 2 yellow, Stage 3 green. Build result: UNSTABLE

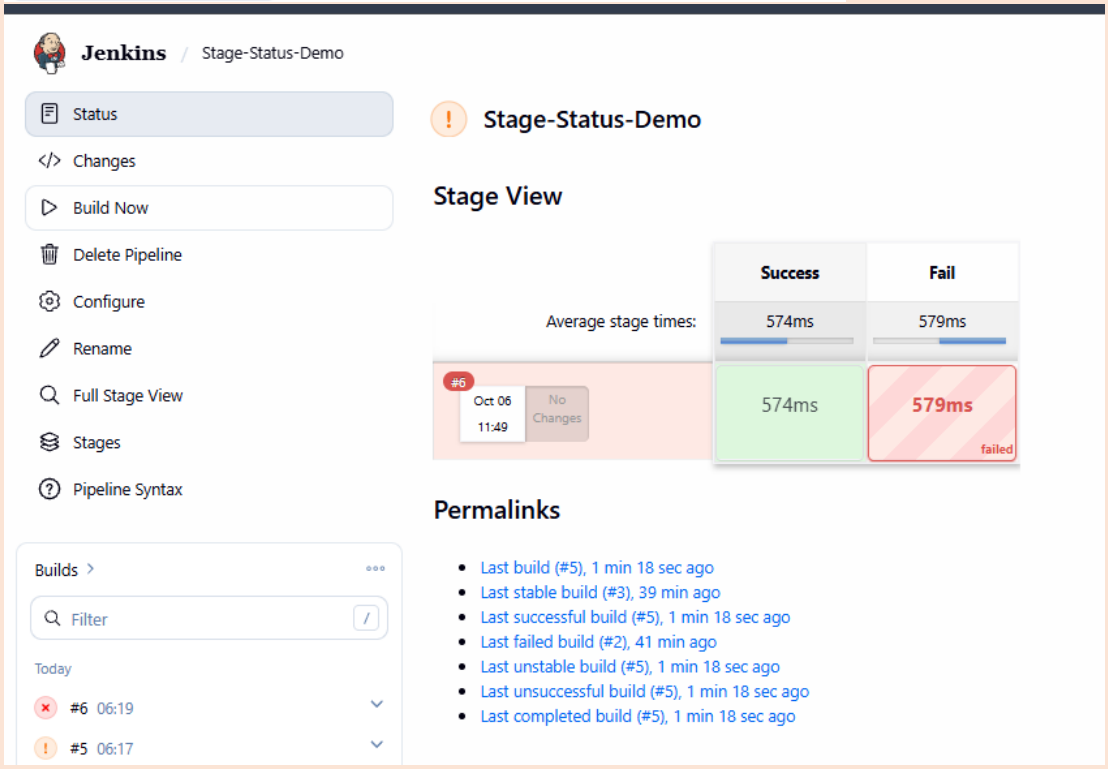
8. Verification

- **Stage View** shows correct colors (green/yellow/red) for each stage
- **Console output** shows Finished: SUCCESS, Finished: UNSTABLE, or Finished: FAILURE
- **Build History** shows blue (SUCCESS), yellow (UNSTABLE), or red (FAILURE) ball
- Skipped stages appear gray with message: *"Stage skipped due to earlier failure(s)"*

Implementation:







9. Conclusion

Stage statuses are not merely visual — they control pipeline flow. FAILED stops the pipeline; UNSTABLE allows it to continue while flagging a problem; SUCCESS ignores failures entirely. warnError provides automatic UNSTABLE behavior for non-critical steps. catchError offers full control over both stage and build status. This is essential for real-world pipelines where linting, reporting, or optional checks should not block deployment.

Task 4: Returning Stdout, Stderr, and Status from Jenkins sh Script

1. Objective

To capture the output (stdout), error messages (stderr), and exit status code of Linux commands executed inside a Jenkins pipeline, and to use these values in Groovy variables for making decisions within the pipeline.

2. Purpose

By default, the Jenkins sh step only prints command output to the console log. It does not allow the pipeline code to read that output. In real-world pipelines, we often need to:

- Read a command's output and use it as a variable (e.g., get a version number)
- Check whether a command succeeded or failed based on its exit code
- Capture error messages to log, alert, or conditionally handle them
- Make decisions (proceed / abort / warn) based on command results

This task explains the three Linux standard streams — **stdout**, **stderr**, and **status code** — and how to return each one from the Jenkins sh step using returnStdout, returnStatus, and shell redirection.

3. Plugins Needed

Feature	Plugin Required
sh step	None — built into Jenkins Pipeline
returnStdout / returnStatus	None — built into Jenkins Pipeline
Shell redirection (2>&1)	None — provided by the Linux shell

No additional plugins are required for this task.

5. Concept Reference

5.1 Standard Streams

Stream	File Descriptor	Purpose	Example
stdout	1	Normal output of a command	echo "hello" → hello
stderr	2	Error messages from a command	ls noSuchFile → ls: cannot access...
status code	—	Exit code: 0 = success, non-zero = failure	echo $? → 0 or 2

5.2 The $? Variable

$? holds the exit status of the last executed command.

```
$ echo "hi"
hi
$ echo $?
0

$ ls noSuchFile
ls: cannot access 'noSuchFile': No such file or directory
```

$ echo $?
2

5.3 The sh Step Flags

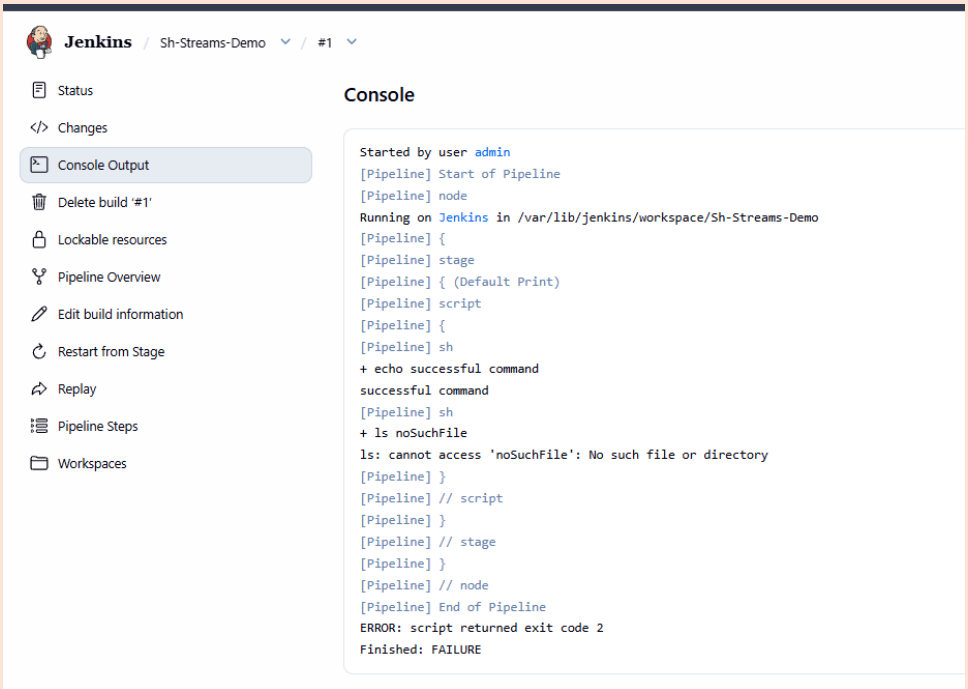
Flag	Returns	Use Case
returnStdout: true	Captures stdout as a string	Read command output into a variable
returnStatus: true	Captures the status code as an integer	Check success/failure without aborting
(no flag for stderr)	—	Must redirect 2>&1 to capture stderr

6. Methods and Step-by-Step Procedures

Method 1: Default Behavior — Print to Console

```
pipeline {
   agent any
   stages {
      stage('Default Print') {
         steps {
            script {
               sh "echo successful command"
               sh "ls noSuchFile"
            }
         }
      }
   }
}
```

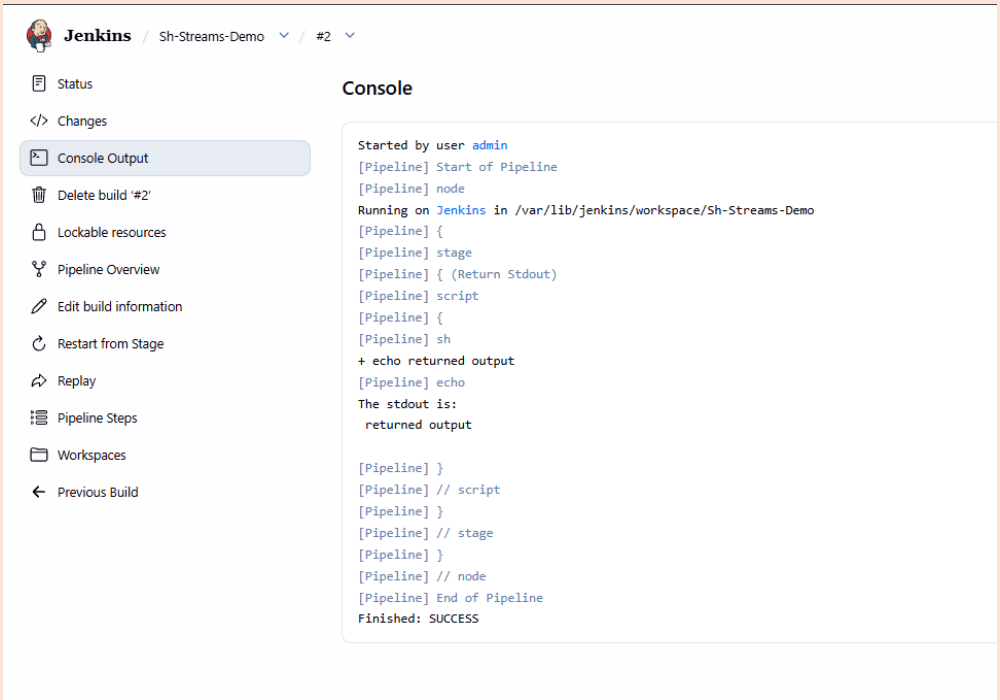
Result: Both commands print to the console. The second fails and the pipeline stops with Finished: FAILURE.



Method 2: Return Stdout

```
pipeline {
    agent any
    stages {
        stage('Return Stdout') {
            steps {
                script {
                    def stdout = sh(
                        script: "echo returned output",
                        returnStdout: true
                    )
                    print "The stdout is:\n ${stdout}"
                }
            }
        }
    }
}
```

Result: The variable stdout contains "returned output\n". Jenkins prints it to the console.



Method 3: Return Status Code

```
pipeline {
    agent any
    stages {
        stage('Return Status') {
            steps {
                script {
                    def status = sh(
                        script: "echo returned output",
                        returnStatus: true
                    )
```

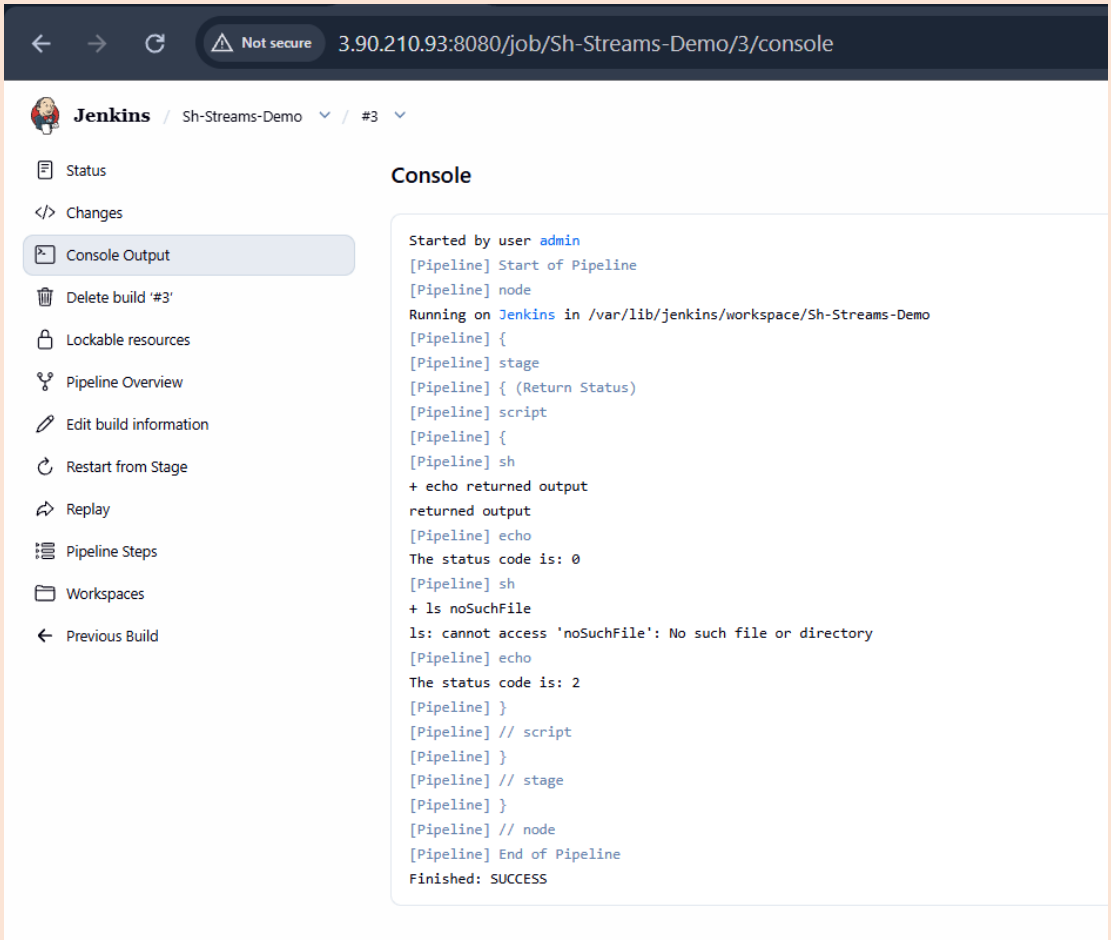
```
            print "The status code is: ${status}"

            status = sh(
               script: "ls noSuchFile",
               returnStatus: true
            )
            print "The status code is: ${status}"
         }
      }
   }
}
```

Result:

- First command → status 0
- Second command → status 2

Note: With returnStatus: true, the pipeline does NOT abort on failure. The status is captured but the pipeline continues.



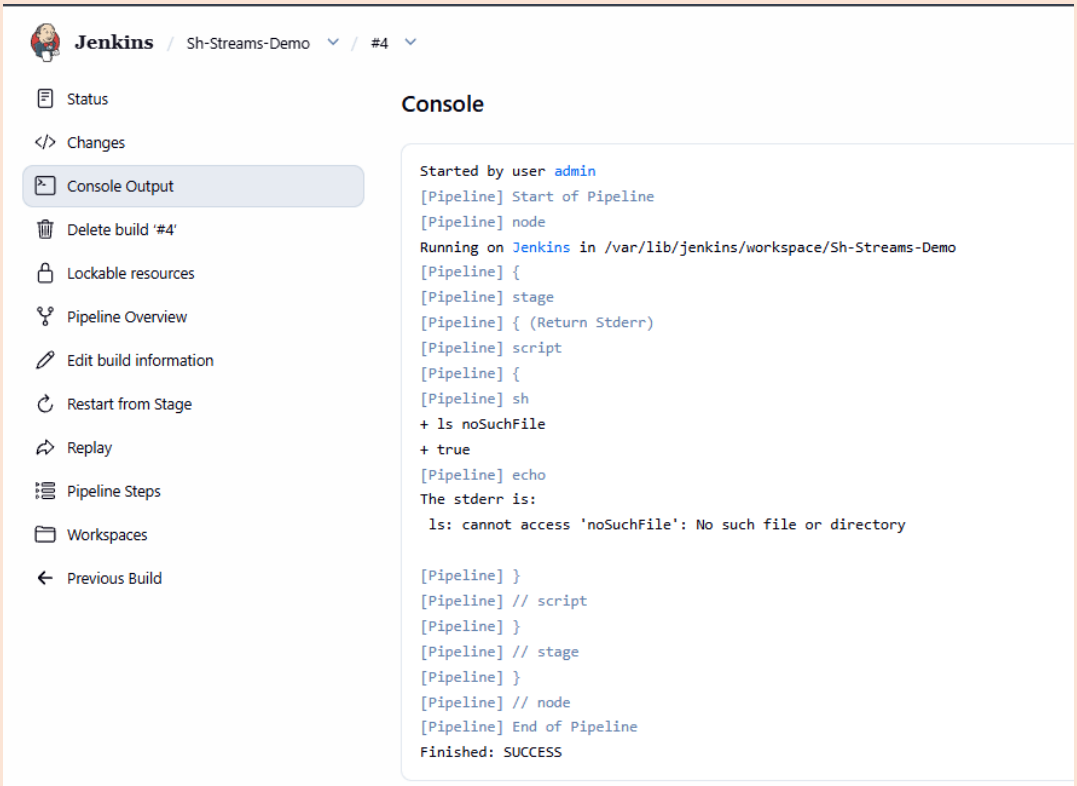
Method 4: Return Stderr (Using Redirection)

```
pipeline {
   agent any
   stages {
      stage('Return Stderr') {
         steps {
            script {
               def stderr = sh(
                  script: "ls noSuchFile 2>&1 || true",
                  returnStdout: true
               )
               print "The stderr is:\n ${stderr}"
            }
         }
      }
   }
}
```

Breakdown of 2>&1 || true**:**

Part	Meaning
2>&1	Redirect file descriptor 2 (stderr) into file descriptor 1 (stdout)
|| true	If the command fails, force a successful exit so Jenkins does not abort

Result: The variable stderr contains the error message: ls: cannot access 'noSuchFile': No such file or directory.



Method 5: Return Both Output and Status in One Step

Stdout + Status:

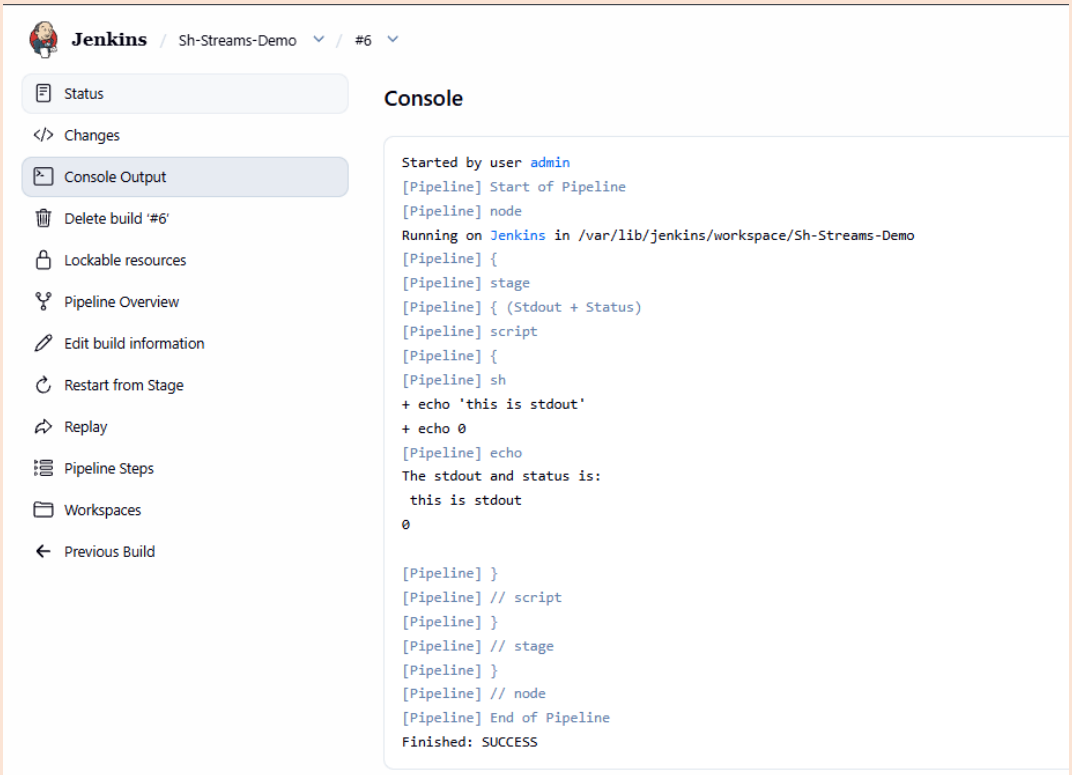
```
pipeline {
    agent any
    stages {
        stage('Stdout + Status') {
            steps {
                script {
                    def result = sh(
                        script: 'echo "this is stdout" && echo "$?"',
                        returnStdout: true
                    )
                    print "The stdout and status is:\n ${result}"
                }
            }
        }
    }
}
```

Result:

text

this is stdout

0

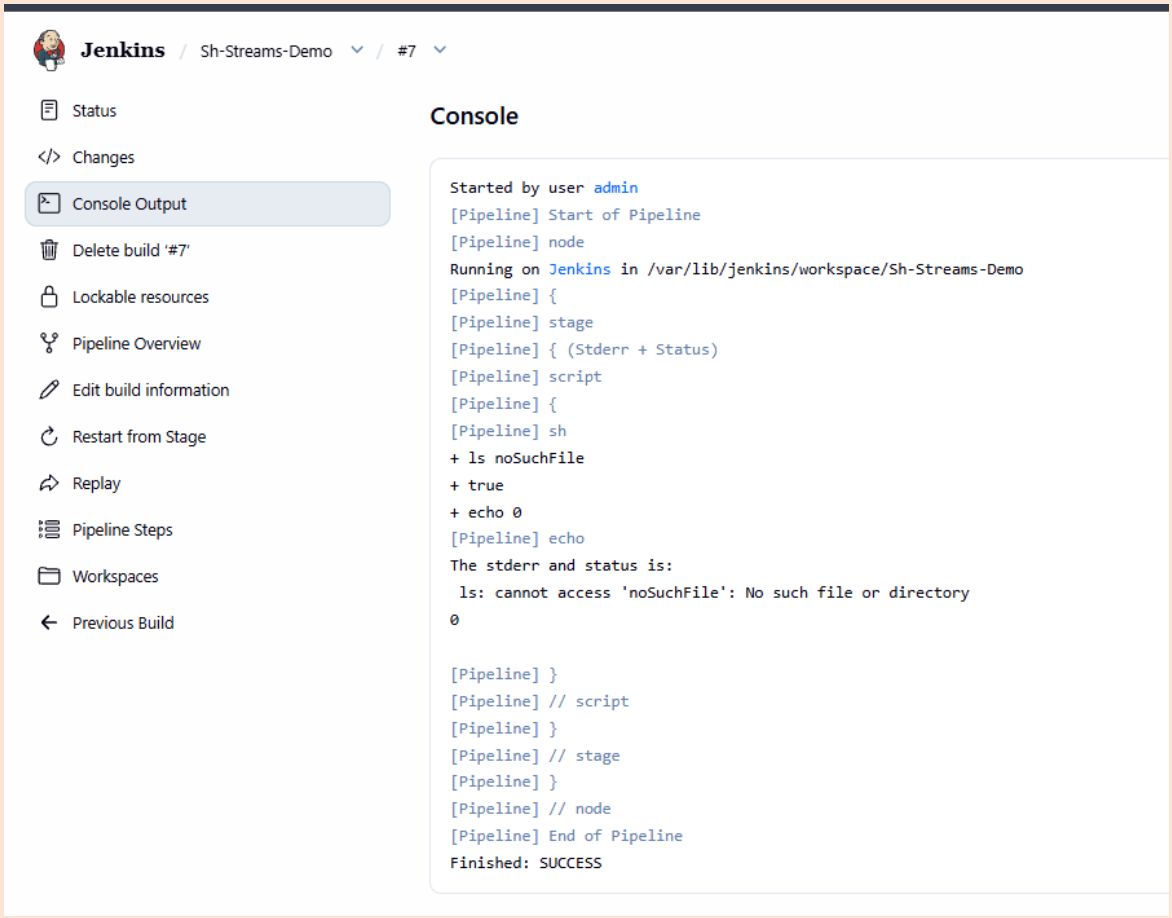


Stderr + Status:

```
pipeline {
   agent any
   stages {
      stage('Stderr + Status') {
         steps {
            script {
               def result = sh(
                  script: 'ls noSuchFile 2>&1 || true && echo "$?"',
                  returnStdout: true
               )
               print "The stderr and status is:\n ${result}"
            }
         }
      }
   }
}
```

Result:

```
text
ls: cannot access 'noSuchFile': No such file or directory
2
```



8. Verification

Run each method and observe:

- **Method 1:** Console shows both outputs; pipeline fails on the second command.
- **Method 2:** stdout variable contains the echoed message.
- **Method 3:** Status codes 0 and 2 printed.
- **Method 4:** Error message captured in the stderr variable.
- **Method 5:** Combined output and status code captured in one variable.
- **Method 6:** Pipeline either continues or aborts based on file presence.

9. Conclusion

The Jenkins sh step provides three powerful flags for capturing Linux command output: returnStdout for stdout, returnStatus for the exit code, and shell redirection (2>&1) for stderr. By capturing these values into Groovy variables, pipelines can make dynamic decisions — checking whether a file exists, validating a command's success, or logging errors for alerts. Understanding standard streams and exit codes is essential for building reliable, self-aware Jenkins pipelines. No additional plugins are required; everything is available in the Jenkins Pipeline core.

Task 5: Updating Jenkins Plugins from the Command Line

1. Objective

To update Jenkins plugins directly from the command line using the Jenkins CLI (jenkins-cli.jar), enabling automation of plugin management without using the Jenkins web dashboard.

2. Purpose

Jenkins administrators often manage multiple Jenkins instances or need to automate routine maintenance. Clicking through the web UI to update plugins is slow and error-prone when repeated across servers. The Jenkins CLI provides two methods — install-plugin and groovy — to list, filter, and update plugins in a scriptable, repeatable way. This is essential for:

- Automating plugin updates across multiple Jenkins servers
- Integrating plugin updates into CI/CD maintenance pipelines
- Reducing manual clicks and human error
- Auditing which plugins are outdated

4. Plugins Needed

Feature	Plugin Required
Jenkins CLI	None — built into Jenkins core
list-plugins command	None — built into Jenkins CLI
install-plugin command	None — built into Jenkins CLI
groovy command	None — built into Jenkins CLI

No additional plugins are required for this task. The Jenkins CLI is a core feature.

5. Concepts Reference

5.1 What is the Jenkins CLI?

The Jenkins CLI is a command-line tool (jenkins-cli.jar) that lets you control Jenkins from a terminal. Anything you can do in the web UI, you can usually do from the CLI — which means you can **automate it**.

5.2 Basic Command Structure

java -jar jenkins-cli.jar -s http://localhost:8080 -auth admin:admin_secret_password <COMMAND>

Article-1

Part	Meaning
java -jar jenkins-cli.jar	Run the CLI tool
-s http://localhost:8080	Jenkins server URL
-auth admin:admin_secret_password	Username and password
<COMMAND>	The action to perform (e.g., list-plugins)

5.3 Key Commands Used

Command	Purpose
list-plugins	Lists installed plugins and shows newer versions in parentheses
install-plugin	Installs or updates specified plugins
groovy	Runs a Groovy script inside Jenkins
-restart	Restarts Jenkins after installation

6. Step-by-Step Procedures

Step 0: Download jenkins-cli.jar

Option A — From Jenkins Web UI:

1. Open Jenkins in browser: http://localhost:8080
2. Go to **Manage Jenkins → Jenkins CLI**
3. Under **Download CLI**, click the link to download jenkins-cli.jar
4. Move it to your working directory (e.g., ~/jenkins-cli/)

Option B — From the CLI endpoint directly:

wget http://3.90.210.93:8080/jnlpJars/jenkins-cli.jar

Verify the download:

ls -lh jenkins-cli.jar

java -jar jenkins-cli.jar -s http://localhost:8080 -auth admin:YOUR_PASSWORD help (if this is not worked) try these

→ java -jar jenkins-cli.jar -s http://localhost:8080 -http -auth admin:<Api Token> help

Run it as follows:

java -jar jenkins-cli.jar -s http://3.90.210.93:8080/ help

If you see the list of available commands, the CLI is working.

Recommended: use an API token

In Jenkins:

Log in as admin.

Click your username in the upper-right.

Open Security / Configure.

Find API Token.

Generate a token.

Copy it immediately.

Article-1

Method 1: install-plugin Command (Direct Way)
Step 1.1: List Installed Plugins
java -jar jenkins-cli.jar -s http://localhost:8080 -auth admin:YOUR_PASSWORD list-plugin
java -jar jenkins-cli.jar -s http://localhost:8080 -http -auth admin:117407dd5085f1e495dec121dba772ca0e list-plugins

Step 1.2: Filter Plugins That Need Updating
➔ printf '%s\n' 'admin:YOUR_TOKEN' > ~/.jenkins-cli-auth
chmod 600 ~/.jenkins-cli-auth
2. Test Jenkins itself
curl -I http://localhost:8080/login

3. Test CLI authentication
```
java -jar jenkins-cli.jar \
  -s http://localhost:8080 \
  -http \
  -auth @/home/ec2-user/.jenkins-cli-auth \
  who-am-i
```

Then run:
```
java -jar jenkins-cli.jar \
  -s http://localhost:8080 \
  -http \
  -auth @/home/ec2-user/.jenkins-cli-auth \
  list-plugins
```

Step 1.3: Update All Plugins in the List
Step 1 — First see what is in $update_list
Run:
echo "$update_list"
If it prints nothing, $update_list is empty.

Method 2: groovy Command (Scripted Way)
Step 2.1: Create the Groovy Script
On your EC2 terminal, run
vi update_plugins.groovy

```
import jenkins.model.*

def plugins = Jenkins.instance.pluginManager.plugins.findAll {
   it.hasUpdate()
}.collect {
   it.getShortName()
}

println "Plugins to update:"
plugins.each {
   println it
}

if (plugins.isEmpty()) {
   println "No plugin updates available."
```

```
} else {
   Jenkins.instance.pluginManager.install(plugins, false).each {
      it.get()
   }

   println "Plugin installation completed."
   Jenkins.instance.safeRestart()
}
```

Now check that the file exists: ls -lh update_plugins.groovy

Step 3 — Make sure your CLI authentication works

```
java -jar jenkins-cli.jar \
 -s http://localhost:8080 \
 -http \
 -auth @/home/ec2-user/.jenkins-cli-auth \
 who-am-i
```

Step 4 — Execute the Groovy script

```
java -jar jenkins-cli.jar \
 -s http://localhost:8080 \
 -http \
 -auth @/home/ec2-user/.jenkins-cli-auth \
 groovy = < update_plugins.groovy
```

Step 5 — Wait for Jenkins to restart

curl -I http://localhost:8080

Method 3: Using an API Token Instead of a Password (Recommended)

Using a password in a command is insecure (visible in shell history). Use an API token instead.

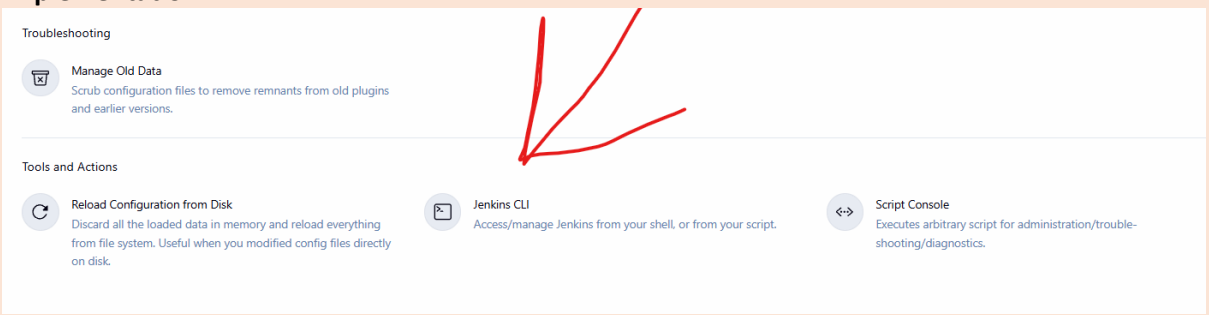
Step 3.1: Generate an API Token

1. In Jenkins, click your username (top right) → **Configure**
2. Under **API Token**, click **Add new Token**
3. Give it a name (e.g., cli-token) → click **Generate**
4. Copy the token (you won't see it again)

Step 3.2: Use the Token in the CLI

```
java -jar jenkins-cli.jar -s http://localhost:8080 -auth admin:YOUR_API_TOKEN list-plugins
```

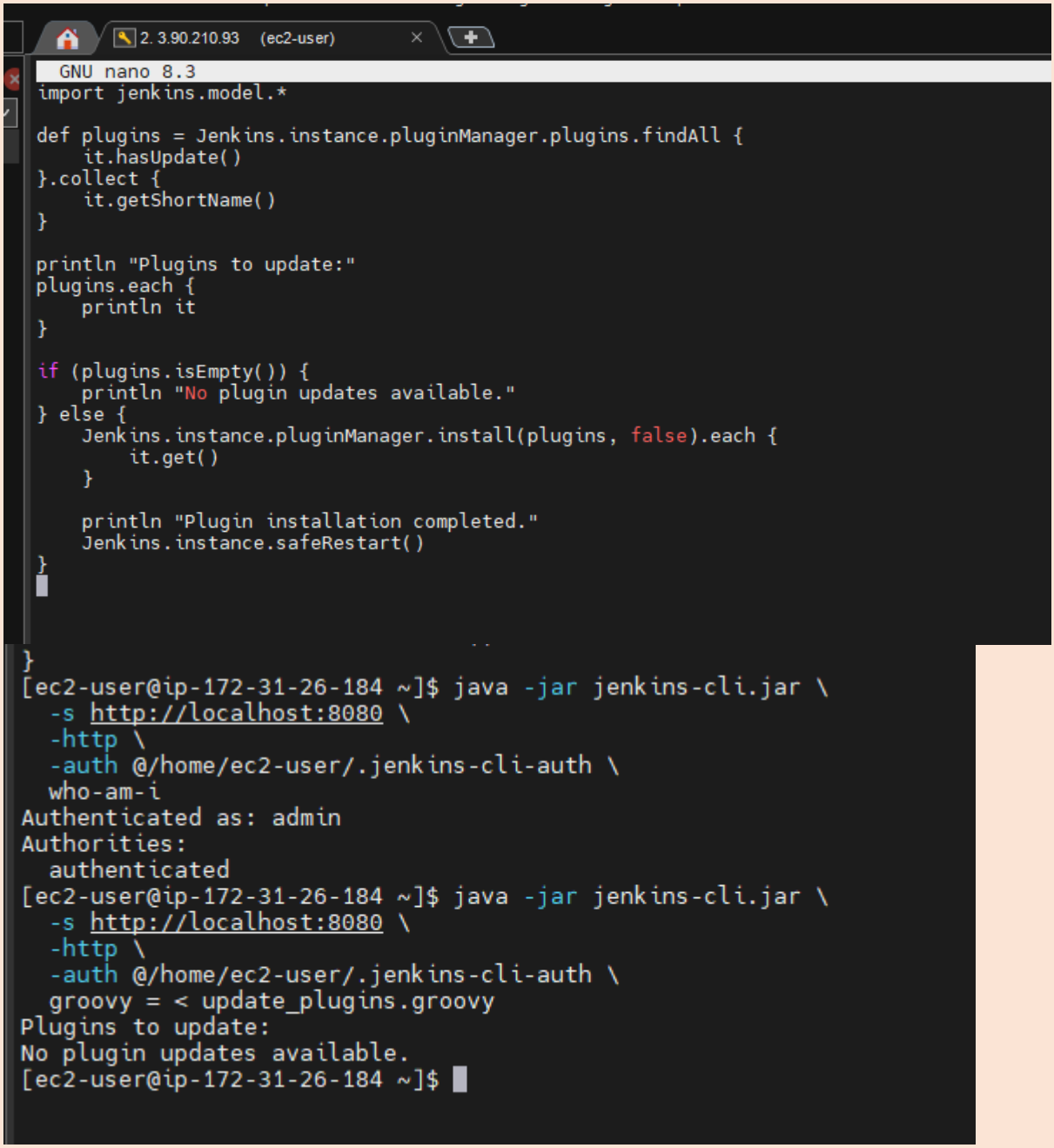
Implementation:



Article-1

```
[ec2-user@ip-172-31-26-184 ~]$
[ec2-user@ip-172-31-26-184 ~]$ java -jar jenkins-cli.jar -s http://localhost:8080 -http -auth admin:117407dd5085f1e495dec121dba772ca0e help
  add-job-to-view
    Adds jobs to view.
  build
    Builds a job, and optionally waits until its completion.
  cancel-quiet-down
    Cancel the effect of the "quiet-down" command.
  clear-queue
    Clears the build queue.
  connect-node
    Reconnect to a node(s)
  console
    Retrieves console output of a build.
  copy-job
    Copies a job.
  create-credentials-by-xml
    Create Credential by XML
  create-credentials-domain-by-xml
    Create Credentials Domain by XML
  create-job
    Creates a new job by reading stdin as a configuration XML file.
  create-node
    Creates a new node by reading stdin as a XML configuration.
  create-view
    Creates a new view by reading stdin as a XML configuration.
  declarative-linter
    Validate a Jenkinsfile containing a Declarative Pipeline
  delete-builds
    Deletes build record(s).
  delete-credentials
    Delete a Credential
  delete-credentials-domain
    Delete a Credentials Domain
  delete-job
    Deletes job(s).
  delete-node
    Deletes node(s)
  delete-view
    Deletes view(s).
  disable-job
    Disables a job.
  disable-plugin
    Disable one or more installed plugins.
  disconnect-node
    Disconnects from a node.
  enable-job
    Enables a job.
  enable-plugin
    Enables one or more installed plugins transitively.
  get-credentials-as-xml
    Get a Credentials as XML (secrets redacted)
  get-credentials-domain-as-xml
    Get a Credentials Domain as XML
  get-gradle
    List available gradle installations
  get-job
```

```
[ec2-user@ip-172-31-26-184 ~]$
[ec2-user@ip-172-31-26-184 ~]$ java -jar jenkins-cli.jar \
  -s http://localhost:8080 \
  -http \
  -auth @/home/ec2-user/.jenkins-cli-auth \
  who-am-i
Authenticated as: admin
Authorities:
  authenticated
[ec2-user@ip-172-31-26-184 ~]$ java -jar jenkins-cli.jar \
  -s http://localhost:8080 \
  -http \
  -auth @/home/ec2-user/.jenkins-cli-auth \
  list-plugins
ant                                 Ant Plugin                                 520.vd082ecfb_16a_9
antisamy-markup-formatter           OWASP Markup Formatter Plugin              173.v680e3a_b_69ff3
apache-httpcomponents-client-4-api Apache HttpComponents Client 4.x API Plugin 4.5.14-269.vfa_2321039a_83
asm-api                             ASM API Plugin                             9.10.1-216.va_9256d3b_844b_
bootstrap5-api                      Bootstrap 5 API Plugin                     5.3.8-1048.va_c299057e35c
bouncycastle-api                    bouncycastle API Plugin                    2.30.1.85.2-304.v4b_5b_62e59a_a_7
branch-api                          Branch API Plugin                          2.1303.v9f3b_95dc329d
build-timeout                       Build Timeout                              1.41
caffeine-api                        Caffeine API Plugin                        3.2.4-208.v7e2da_a_7db_82b_
checks-api                          Checks API plugin                          422.vc7e8b_51488c6
cloudbees-folder                    Folders Plugin                             6.1106.v3a_d9a_6d2465e
commons-lang3-api                   commons-lang3 v3.x Jenkins API Plugin      3.21.0-122.v6a_ea_4fd34049
commons-text-api                    commons-text API Plugin                    1.15.0-218.va_61573470393
credentials-binding                 Credentials Binding Plugin                 728.v902a_273b_8947
credentials                         Credentials Plugin                         1511.v2e3cb_0008ef0
dark-theme                          Dark Theme                                 652.vea_da_dfea_e769
display-url-api                     Display URL API                            2.217.va_6b_de84cc74b_
durable-task                        Durable Task Plugin                        686.v80ff80875b_82
echarts-api                         ECharts API Plugin                         6.1.0-1321.v86c4c5114494
eddsa-api                           EDDSA API Plugin                           0.3.0.1-29.v67e9a_1c969b_b_
email-ext                           Email Extension Plugin                     2038.v7b_8817a_499d9
font-awesome-api                    Font Awesome API Plugin                    7.3.1-1013.v0835a_879ec6d
git-client                          Git client plugin                          6.6.1
git                                 Git plugin                                 5.10.1
github-api                          GitHub API Plugin                          1.330-492.v3941a_032db_2a_
github-branch-source                GitHub Branch Source Plugin                1983.vfa_27ed961853
github                              GitHub plugin                              1.47.0
gradle                              Gradle Plugin                              2.20.1253.vc116f0763a_eb_
gson-api                            Gson API Plugin                            2.14.0-201.v8eefe5515533
instance-identity                   Instance Identity                          203.v15e81a_1b_7a_38
ionicons-api                        Ionicons API                               94.vcc3065403257
jackson-annotations2-api            Jackson Annotations 2 API Plugin           2.22-19.v10a_a_582ea_26e
jackson2-api                        Jackson 2 API Plugin                       2.22.2-445.vdc613f1d8012
```



Conclusion

The Jenkins CLI (jenkins-cli.jar) provides a powerful, scriptable way to manage plugins without the web UI. Two methods were demonstrated: the install-plugin command for direct updates and the groovy command for running custom scripts. Both methods use list-plugins to identify outdated plugins and -restart (or safeRestart()) to apply changes. Using an API token instead of a password improves security. This approach is ideal for administrators managing multiple Jenkins instances, enabling automation of routine maintenance tasks that would otherwise require manual clicking in the dashboard. No additional plugins are required — the Jenkins CLI is part of Jenkins core.

Article-1

Task 6: Cleaning Jenkins Workspace After Each Build

1. Objective

To clean the Jenkins workspace — the directory where a job stores its files — before and/or after each build using three different methods: deleteDir(), cleanWs(), and the native Linux rm command.

2. Purpose

Jenkins reuses the same workspace directory for successive builds of the same job. Files from previous builds can linger and cause:

- **Conflicting files** — old outputs mixed with new ones
- **Stale artifacts** — builds that pass only because of leftover files ("works on my machine" syndrome)
- **Disk space bloat** — accumulated files over many builds
- **Unpredictable results** — hard-to-debug failures caused by stale state

Cleaning the workspace ensures every build starts from a known, fresh state. This is essential for reliable, reproducible CI/CD pipelines.

4. Plugins Needed

Method	Plugin Required	Notes
deleteDir()	None — built into Jenkins core (Workflow Basic Steps)	Comes pre-installed
cleanWs()	**Workspace Cleanup Plugin**	Must be installed manually
rm *	None — uses native Linux shell	No plugin needed

5. Concepts Reference

A workspace is a directory on the node where a job runs. It stores all files related to the build: source code, compiled artifacts, temporary files, logs, etc.

Path structure: /var/lib/jenkins/workspace/<folder-name>/<job-name>/

5.2 Why Each Job Has Its Own Workspace

Each Jenkins job gets a separate workspace directory named after the job. This isolates jobs from each other. However, successive builds of the **same** job reuse the **same** workspace — which is why cleaning is needed

6. Step-by-Step Procedures

Step 0: Create the Job

1. Click **New Item** → Name: Workspace-Cleanup-Demo → Select **Pipeline** → **OK**

Method 1: deleteDir() — Built-in Step

```
pipeline {
  agent any
  stages {
    stage("Create File") {
      steps {
        script {
          sh "touch myfile.txt"
          echo "File created."
        }
      }
    }
    stage("Clean with deleteDir") {
      steps {
```

```
            script {
                sh "ls -la"
                deleteDir()
                sh "ls -la"
            }
        }
    }
  }
}
```

Step 1.2: Click **Save** → Click **Build Now**.

Result: The file myfile.txt is present before deleteDir() and gone after. The workspace is empty.
Note: deleteDir() is built into Jenkins core (Workflow Basic Steps plugin, pre-installed). No installation required.

Method 2: cleanWs() — Workspace Cleanup Plugin
Step 2.1: Install the Plugin

1. Go to **Manage Jenkins → Plugins**
2. Click the **Available** tab
3. In the search box, type: Workspace Cleanup
4. Check the box next to **Workspace Cleanup Plugin**
5. Click **Install** (or **Install without restart**)
6. Wait for installation to complete

- **In Pipelines:** It provides the cleanWs() step to completely wipe the workspace. [1, 2]
- **In Freestyle Jobs:** It adds the "Delete workspace before build starts" wrapper and the "Delete workspace when build is done" post-build action. [1, 2]

Step 2.2: Use the Step

```
pipeline {
  agent any
  stages {
    stage("Create File") {
      steps {
        script {
          sh "touch myfile.txt"
          echo "File created."
        }
      }
    }
    stage("Clean with cleanWs") {
      steps {
        script {
          sh "ls -la"
          cleanWs()
          sh "ls -la"
        }
      }
    }
  }
}
```

Click **Save** → Click **Build Now**.
Result: Same effect as deleteDir() — workspace is empty. But cleanWs() is more powerful (can clean before and after, supports patterns, etc.).

Method 3: Native Linux rm Command

```
pipeline {
   agent any
   stages {
      stage("Create File") {
         steps {
            script {
               sh "touch myfile.txt"
               echo "File created."
            }
         }
      }
      stage("Clean with rm") {
         steps {
            script {
               sh "ls -la"
               sh "rm -rf *"
               sh "ls -la"
            }
         }
      }
   }
}
```

Result: rm -rf * deletes everything in the current directory. The workspace is empty.
Warning: rm -rf * is powerful and irreversible. Use with caution. rm * (without -rf) only deletes files, not directories.

Method 4: Clean Before AND After Build (Best Practice)
Use the post { always { cleanWs() } } block to always clean after the build, regardless of outcome.

```
pipeline {
   agent any
   stages {
      stage("Clean Before") {
         steps {
            cleanWs()
            echo "Workspace cleaned before build."
         }
      }
      stage("Build") {
         steps {
            sh "echo 'building...' > build.log"
            sh "ls -la"
         }
      }
      stage("Test") {
         steps {
```

```
            sh "echo 'testing...' >> build.log"
            sh "ls -la"
        }
    }
  }
  post {
    always {
      cleanWs()
      echo "Workspace cleaned after build."
    }
  }
}
```

Result:

1. Workspace cleaned at start
2. Build creates build.log
3. Test appends to build.log
4. Workspace cleaned again after build — **always**, even if a stage fails

7. Comparison Table

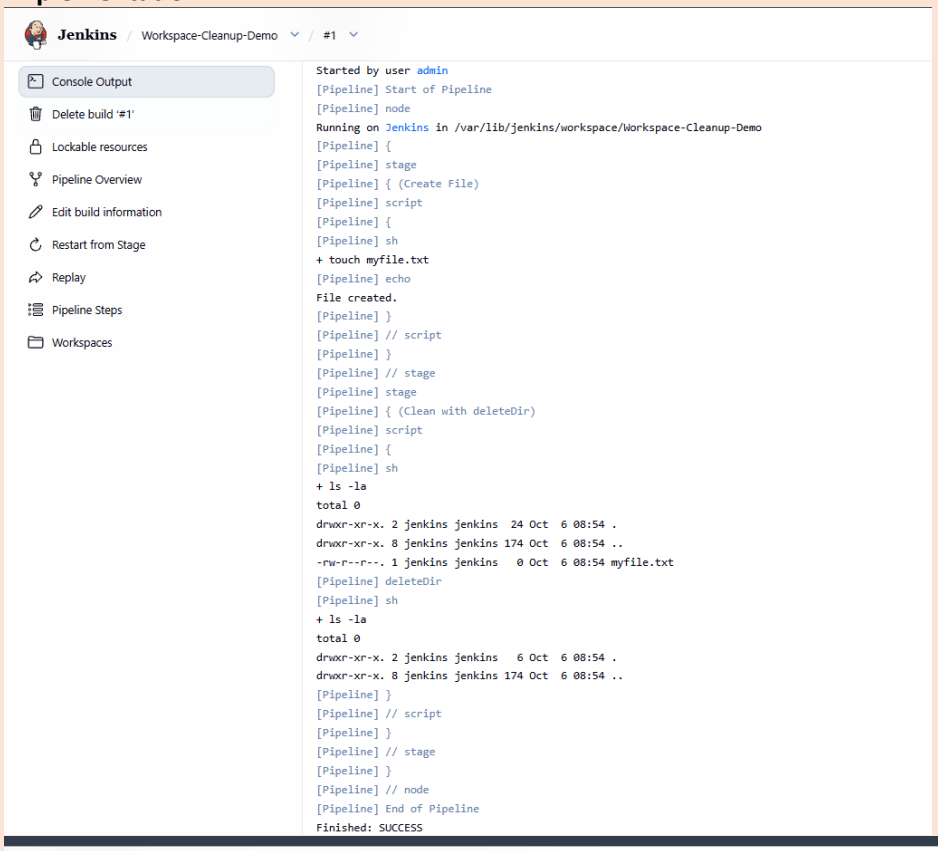
Method	Plugin Required	Deletes Directories?	Best For
deleteDir()	❌ No (built-in)	✅ Yes	Simple one-off cleanup
cleanWs()	✅ Workspace Cleanup Plugin	✅ Yes	Advanced cleanup, before/after build
rm -rf *	❌ No (shell)	✅ Yes	Quick manual cleanup
post { always { cleanWs() } }	✅ Workspace Cleanup Plugin	✅ Yes	Guaranteed cleanup after every build

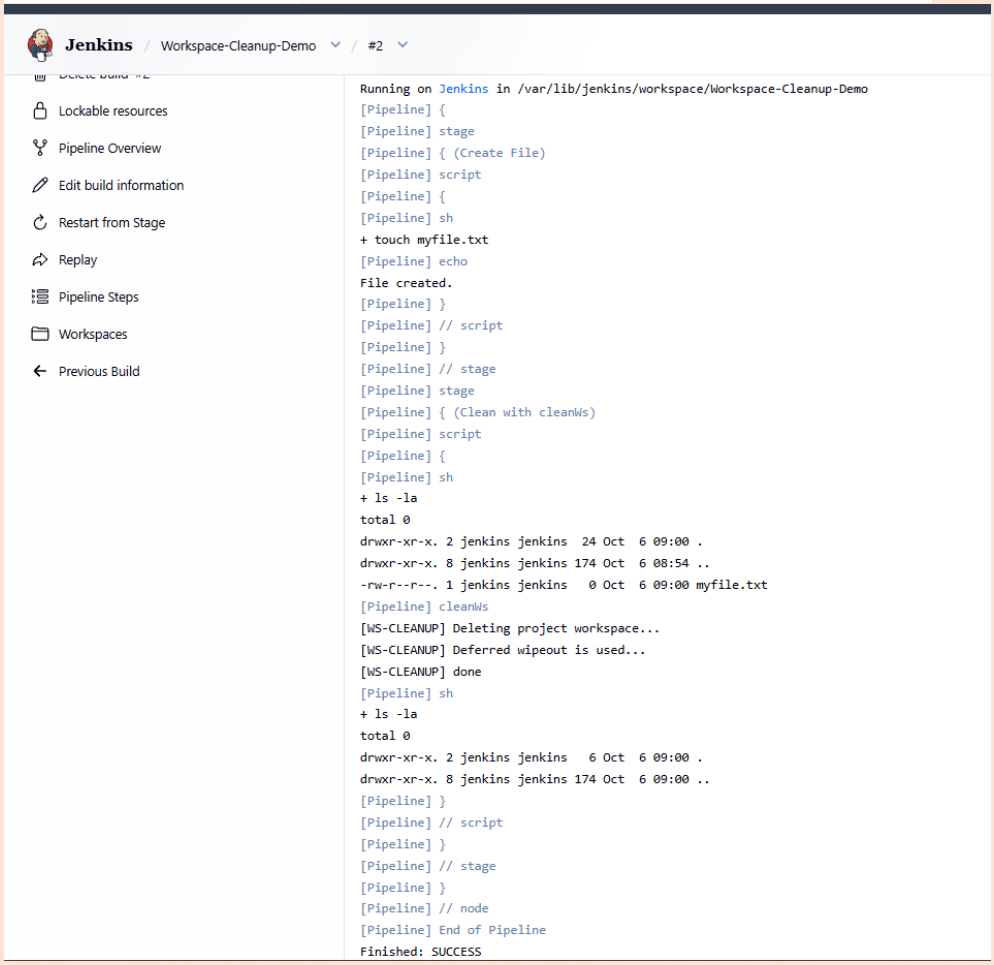
9. Common Pitfalls and Solutions

Problem	Cause	Solution
deleteDir() not recognized	Old Jenkins version	Jenkins 2.580.1 has it built-in — no plugin needed
cleanWs() not recognized	Plugin not installed	Install **Workspace Cleanup Plugin**
rm * doesn't delete hidden files	* doesn't match dotfiles	Use rm -rf .* * (careful)
Workspace not empty after cleanup	Files locked by another process	Stop running builds first
Cleanup deletes files you needed	Cleaning after build	Move cleanup to **before** the build instead
Build fails after cleanWs()	Cleaned workspace mid-pipeline	Place cleanup at start or in post block only

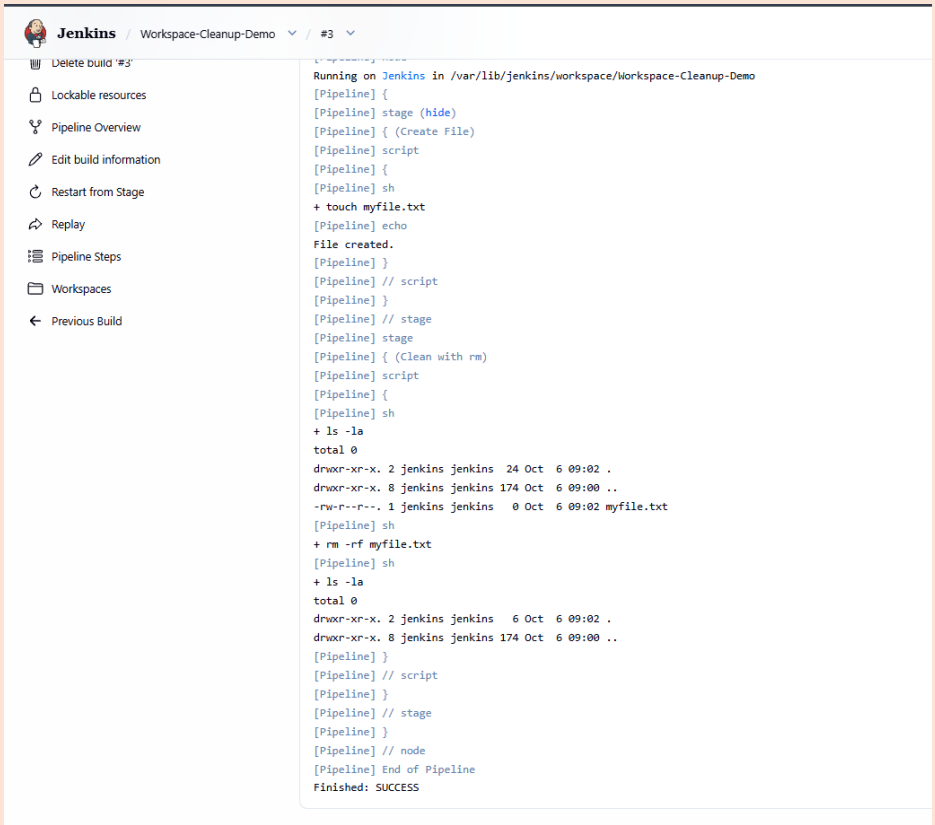
Article-1

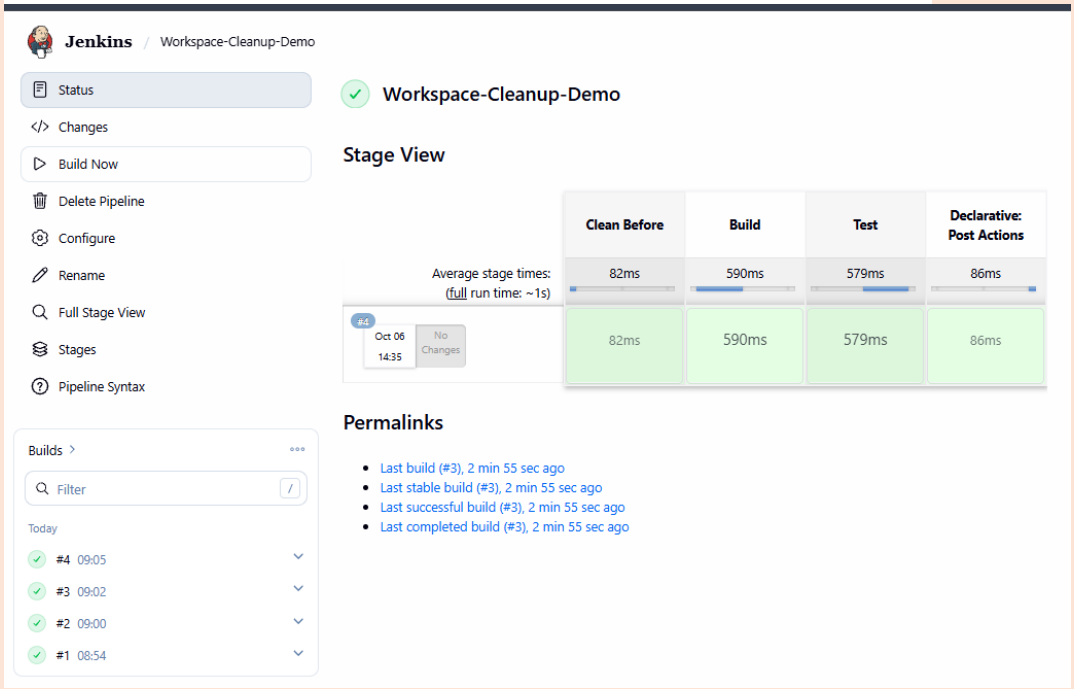
Implementation:





Article-1





Conclusion:

Cleaning the Jenkins workspace ensures reliable, reproducible builds by preventing stale files and conflicts.

You can use three methods:

- **deleteDir()**: Built-in, simple, and requires no plugins.
- **cleanWs()**: Requires the *Workspace Cleanup Plugin* but offers advanced control.
- **rm -rf ***: A native Linux command that needs no plugins.

Best Practice: Clean before the build for a fresh start, and use post { always { cleanWs() } } to guarantee cleanup after the build completes, regardless of its outcome.

Article-1

Task 7: Creating JSON Strings from Groovy Variables in Jenkins Pipeline

1. Objective
To serialize Groovy objects (maps, lists, strings) into JSON strings inside a Jenkins pipeline using two methods: the Groovy JsonOutput class and the Jenkins-native writeJSON step.

2. Purpose
Serialization is the process of converting an object into a standard format (like JSON) that can be stored, transmitted, and reconstructed elsewhere. In Jenkins pipelines, JSON is essential for:

- **Sending build data to APIs** — most REST APIs expect JSON payloads
- **Storing build metadata** — save job status, commit info, or test results to a file
- **Passing data between pipeline stages** — serialize in one stage, read in the next
- **Generating configuration files** — write JSON configs for deployment tools (Ansible, Terraform, Docker)
- **Integrating with external systems** — Slack, Jira, ServiceNow, and others accept JSON

Without JSON serialization, passing structured data between pipeline stages or to external systems requires fragile string manipulation. This task solves that by using proper serialization tools

4. Plugins Needed

Method	Plugin Required	Notes
JsonOutput (Groovy)	None — part of Groovy runtime included with Jenkins	No installation needed
writeJSON (Jenkins step)	**Pipeline Utility Steps Plugin**	Must be installed manually

5. Concepts Reference
5.1 What Is a Groovy Variable?
in Jenkins pipelines, you can create variables inside script { } blocks using the def keyword:
`def student = ["name":"John", "age":20, "subjects":["English", "Math", "Physics"]]`

This creates a LinkedHashMap — a key-value data structure. Values can be strings, numbers, or even nested lists.
5.2 What Is JSON?
JSON (JavaScript Object Notation) is a universal text format for representing structured data. It uses key-value pairs and arrays

```
{
   "name": "John",
   "age": 20,
   "subjects": ["English", "Math", "Physics"]
}
```

Why JSON matters: APIs, config files, databases, and message queues all understand JSON. It is the "common language" for data exchange

6. Step-by-Step Procedures

Step 0: Create the Job

1. Click New Item → Name: Json-Serialization-Demo → Select Pipeline → OK

Method 1: JsonOutput — Groovy's Built-in Class

```
import groovy.json.JsonOutput

pipeline {
   agent any
   stages {
      stage("Create and Serialize Groovy Variable") {
         steps {
            script {
               def student = [
                  "name": "John",
                  "age": 20,
                  "subjects": ["English", "Math", "Physics"]
               ]

               echo "Variable type: ${student.getClass()}"
               echo "Name: ${student.name}"
               echo "Age: ${student.age}"
               echo "First subject: ${student.subjects[0]}"

               def json_string = JsonOutput.toJson(student)
               def json_pretty = JsonOutput.prettyPrint(json_string)
               echo "JSON output:\n${json_pretty}"
            }
         }
      }
   }
}
```

Step 1.2: Click **Save** → Click **Build Now**.

Breakdown:

Line	Purpose
import groovy.json.JsonOutput	Import the JSON serialization class
def student = [...]	Create a Groovy map (LinkedHashMap)
JsonOutput.toJson(student)	Convert the map to a compact JSON string
JsonOutput.prettyPrint(json_string)	Format the JSON with indentation for readability

Method 2: writeJSON — Jenkins-Native Step
Step 2.1: Install the Pipeline Utility Steps Plugin

1. Go to **Manage Jenkins → Plugins**
2. Click the **Available** tab
3. Search for: Pipeline Utility Steps
4. Check the box next to **Pipeline Utility Steps Plugin**
5. Click **Install** (or **Install without restart**)
6. Wait for installation to complete

Step 2.2: Update the pipeline script:

```
pipeline {
  agent any
  stages {
    stage("Serialize with writeJSON") {
      steps {
        script {
          def student = [
            "name": "John",
            "age": 20,
            "subjects": ["English", "Math", "Physics"]
          ]

          def json_pretty = writeJSON(
            json: student,
            returnText: true,
            pretty: 4
          )
          echo "JSON output:\n${json_pretty}"
        }
      }
    }
  }
}
```

Step 2.3: Click **Save** → Click **Build Now**.
Step 2.4: Observe the console output — same pretty JSON as Method 1.

Breakdown of writeJSON **parameters:**

Parameter	Purpose
json: student	The Groovy object to serialize
returnText: true	Return the JSON as a string (instead of writing to a file)
pretty: 4	Indent using 4 spaces per level

Note: If returnText is false (default) and file is specified, writeJSON writes the JSON directly to a file in the workspace.

Method 3: Write JSON to a File (Real-World Pattern)

```
pipeline {
    agent any
    stages {
        stage("Write JSON to File") {
            steps {
                script {
                    def buildInfo = [
                        "job": env.JOB_NAME,
                        "build": env.BUILD_NUMBER,
                        "node": env.NODE_NAME,
                        "status": "success"
                    ]

                    writeJSON(
                        file: "build-info.json",
                        json: buildInfo,
                        pretty: 4
                    )

                    echo "JSON file written."
                    sh "cat build-info.json"
                }
            }
        }
    }
}
```

Real-world use: Save this JSON as an artifact, send it to an API, or read it in a later stage.

Method 4: Pass JSON Between Stages

```
import groovy.json.JsonOutput

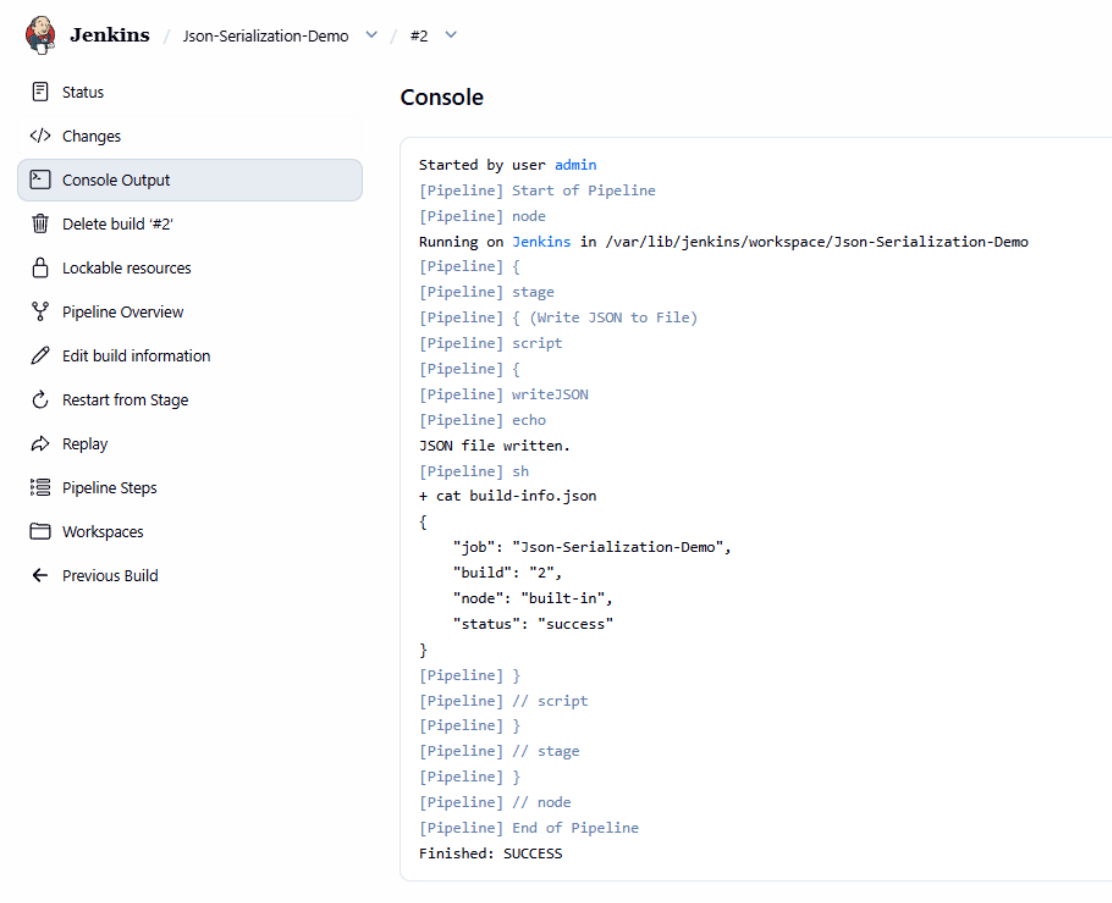
pipeline {
    agent any
    stages {
        stage("Stage 1 - Produce JSON") {
            steps {
                script {
                    def data = ["project": "Demo", "version": "1.0.0"]
                    def jsonStr = JsonOutput.toJson(data)
                    writeFile file: "data.json", text: jsonStr
                    echo "JSON written: ${jsonStr}"
                }
            }
        }
        stage("Stage 2 - Consume JSON") {
            steps {
                script {
                    def content = readFile("data.json")
                    def parsed = readJSON(text: content)
```

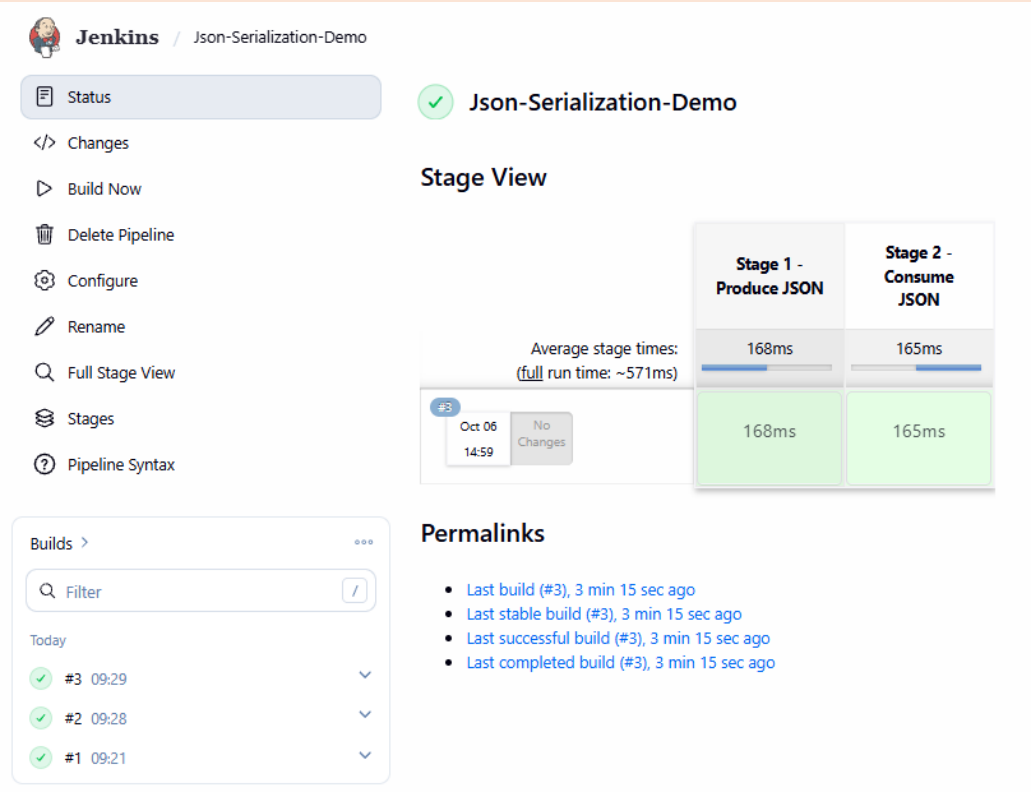
```
            echo "Project: ${parsed.project}"
            echo "Version: ${parsed.version}"
        }
    }
  }
 }
}
```

7. Comparison Table

Feature	JsonOutput	writeJSON
Plugin required	❌ No (Groovy runtime)	✅ Yes (Pipeline Utility Steps)
Groovy import required	✅ Yes (import groovy.json.JsonOutput)	❌ No
Can write directly to file	❌ (must use writeFile separately)	✅ (with file: parameter)
Pretty printing	✅ (prettyPrint())	✅ (pretty: N)
Output format	Identical JSON	Identical JSON
Best for	Avoiding plugin installs	Jenkins-native style

Implementation:





Conclusion:
Serializing Groovy variables to JSON is essential for external system integration and structured pipeline data flow.
You can use two main methods to produce identical JSON output:

- **JsonOutput class**: Groovy-native and ideal for avoiding plugin dependencies.

- **writeJSON step**: Jenkins-native, requiring the *Pipeline Utility Steps* plugin, and cleanly integrated with Jenkins file operations.

Real-World Patterns: Use these to write JSON files for build artifacts, pass structured data between stages using writeFile/readFile/readJSON, and prepare JSON payloads for API calls

Task 8: Fixing Jenkins "Pipeline sh bad substitution" Error
1. Objective
To understand, diagnose, and fix the Jenkins pipeline error Bad substitution that occurs when shell (sh) steps incorrectly handle variable interpolation, and to apply multiple proven solutions.
2. Purpose
The Bad substitution error is one of the most common and confusing errors in Jenkins pipelines. It occurs when the shell cannot replace a variable (like ${env.BUILD_NUMBER}) with its value. This error:

- **Halts pipeline execution** — the affected stage fails, blocking all subsequent stages
- **Delays deployments** — builds cannot complete until the issue is resolved
- **Wastes troubleshooting time** — the error message is cryptic and the root cause is often non-obvious
- **Occurs silently in certain quote styles** — developers often don't realize single quotes disable substitution

4. Plugins Needed

Feature	Plugin Required
sh step	None — built into Jenkins Pipeline
withEnv block	None — built into Jenkins Pipeline
Environment directive	None — built into Jenkins Pipeline
Groovy string handling	None — part of Groovy runtime

5. Concepts Reference

5.1 What Does "Bad Substitution" Mean?

The shell script is trying to replace a variable (e.g., ${env.BUILD_NUMBER}) but the syntax or context is wrong, so the shell gives up with:

```
script.sh: Bad substitution
ERROR: script returned exit code 2
```

5.2 Root Causes

Cause	Description
Wrong quote type	Single quotes ('...') disable variable substitution
Wrong shell	sh does not support all syntax that bash does
Variable not exported	Shell cannot see Jenkins/Groovy variables unless exported

5.3 Groovy vs Shell Quoting Rules

Quote Type	Substitution?	Multi-line?	Escaping needed?
'single'	❌ No	❌ No	None (literal)
"double"	☑ Yes	❌ No	Yes, for internal "
'''triple single'''	❌ No	☑ Yes	None (literal)
"""triple double"""	☑ Yes	☑ Yes	No (clean)

6. Step-by-Step Procedures

1. Click **New Item** → Name: Bad-Substitution-Demo → Select **Pipeline** → **OK**

Step 1: Reproduce the Error

```
pipeline {
  agent any
  environment {
    buildDir = "/var/lib/jenkins/workspace/Bad-Substitution-Demo/"
  }
  stages {
    stage('Reproduce Error') {
```

```
        steps {
            sh 'echo ${env.BUILD_NUMBER}'
        }
    }
  }
}
```

Result: The error is reproduced. The single quotes prevented Groovy from substituting the variable, and the shell could not handle ${env.BUILD_NUMBER} syntax (that's Groovy syntax, not shell syntax).

Diagnosis: The shell received the literal text ${env.BUILD_NUMBER}. Shell syntax for variables is $VAR or ${VAR} — but env.BUILD_NUMBER with a dot is not valid shell syntax.

Fix 1: Use Double Quotes

```
pipeline {
  agent any
  environment {
    buildDir = "/var/lib/jenkins/workspace/Bad-Substitution-Demo/"
  }
  stages {
    stage('Fix with Double Quotes') {
      steps {
        sh "echo ${env.BUILD_NUMBER}"
      }
    }
  }
}
```

Why it works: Groovy substitutes ${env.BUILD_NUMBER} with the actual build number **before** the shell sees the command. The shell then receives echo 5 — perfectly valid.
Gotcha: If the command contains its own double quotes, escape them with \":

```
sh "curl \"http://example.com/${env.BUILD_NUMBER}\""
```

Fix 2: Use Triple Double Quotes ("""...""")

```
pipeline {
  agent any
  stages {
    stage('Fix with Triple Quotes') {
      steps {
        sh """echo "Build number is: ${env.BUILD_NUMBER}"
              echo "Job name is: ${env.JOB_NAME}"
              echo "Node is: ${env.NODE_NAME}" """
      }
    }
  }
}
```

Why it works:

- Groovy substitutes ${env.*} variables before passing to shell
- Internal double quotes are **not** escaped — cleaner code
- Multi-line commands are handled naturally
- No need for \n or concatenation

Fix 3: Force Bash Shell with Shebang

```
pipeline {
  agent any
  environment {
    buildDir = "/var/lib/jenkins/workspace/Bad-Substitution-Demo/"
  }
  stages {
    stage('Fix with Bash Shebang') {
      steps {
        sh '''#!/bin/bash -xe
        echo "Running in bash"
        echo "Build dir is: $buildDir"
        echo "Build number is: $BUILD_NUMBER"
        '''
      }
    }
  }
}
```

Breakdown of #!/bin/bash -xe**:**

Part	Meaning
#!/bin/bash	Use bash instead of sh
-x	Print each command before executing (debug)
-e	Exit immediately if any command fails

Important: With '''...''' (single triple quotes), Groovy does **NOT** substitute. The shell handles substitution instead. But this only works because buildDir and BUILD_NUMBER are **exported as environment variables** by Jenkins — the shell can see them directly

Fix 4: Use withEnv to Export Variables

```
pipeline {
  agent any
  environment {
    buildDir = "/var/lib/jenkins/workspace/Bad-Substitution-Demo/"
  }
  stages {
    stage('Fix with withEnv') {
      steps {
        withEnv(["MY_BUILD=${env.BUILD_NUMBER}"]) {
          sh '''#!/bin/bash -xe
          echo "Build number from withEnv: $MY_BUILD"
          echo "Build dir from environment: $buildDir"
```

```
            '''
          }
        }
      }
    }
  }
}
```

Why it works: withEnv explicitly exports MY_BUILD to the shell environment. The shell can now reference $MY_BUILD directly.
When to use: When you want the **shell** to do the substitution, not Groovy. This is useful when the variable must not be pre-substituted (e.g., you want the value at shell runtime).

7. Comparison Table of Fixes

Fix	When to Use	Pros	Cons
Double quotes "..."	Simple commands	Easy to read	Escaping internal "
Triple double quotes """..."""	Multi-line commands with internal quotes	No escaping needed	Slightly unusual syntax
Bash shebang #!/bin/bash	When sh lacks syntax support	Better compatibility + debug	Requires shell knowledge
withEnv	Exporting Jenkins vars to shell	Explicit control	More verbose
Combination	Complex, production pipelines	Maximum robustness	More complex code

9. Debugging Tips

Tip	How
Isolate the issue	Strip the command to just sh 'echo ${env.BUILD_NUMBER}'
Check quote type	Single quotes = no substitution; double/triple = substitution
Add -x	Use #!/bin/bash -xe to see each command before execution
Echo before running	Add sh 'echo "would run: command"' to preview
Print workspace	sh 'pwd' to confirm current directory
Build incrementally	Start with a simple command, add complexity step by step

10. Common Pitfalls and Solutions

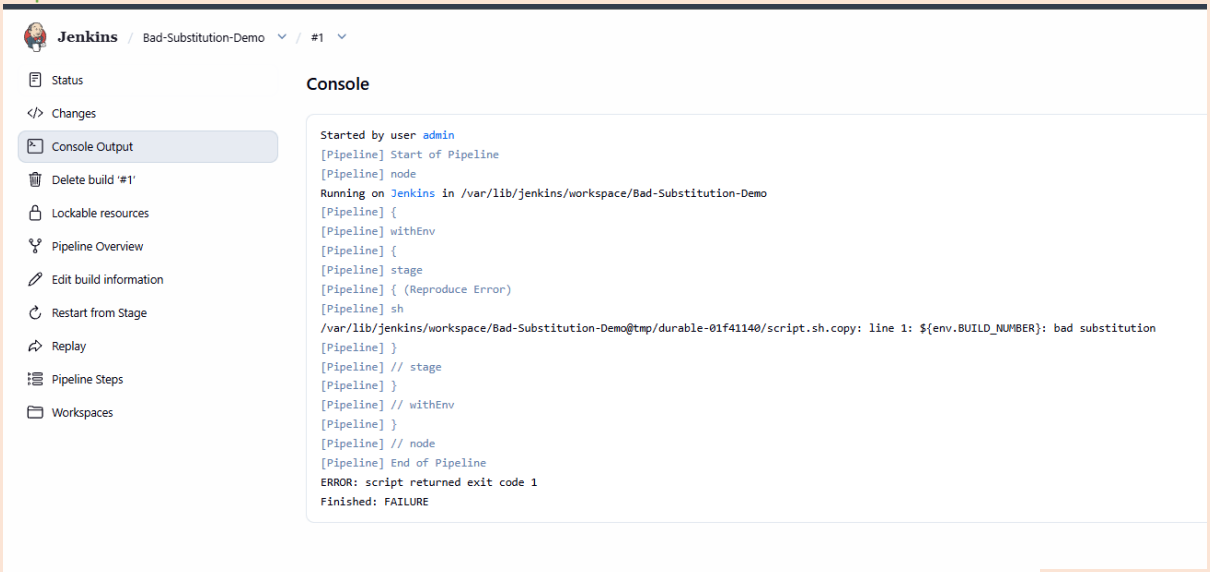
Problem	Cause	Solution
Bad substitution with ${env.X}	Single quotes	Switch to double quotes
Command has internal " breaks	Using "..." with unescaped quotes	Use """...""" or escape with \"
Variable empty inside shell	Not exported	Use withEnv
Shebang line appears in output	'''...''' doesn't process shebang correctly	Ensure shebang is the first line inside the quotes
-e flag stops on non-zero exit	Expected behavior	Remove -e or wrap command with || true
Groovy sandbox blocks withEnv	Script approval needed	Approve or disable sandbox temporarily

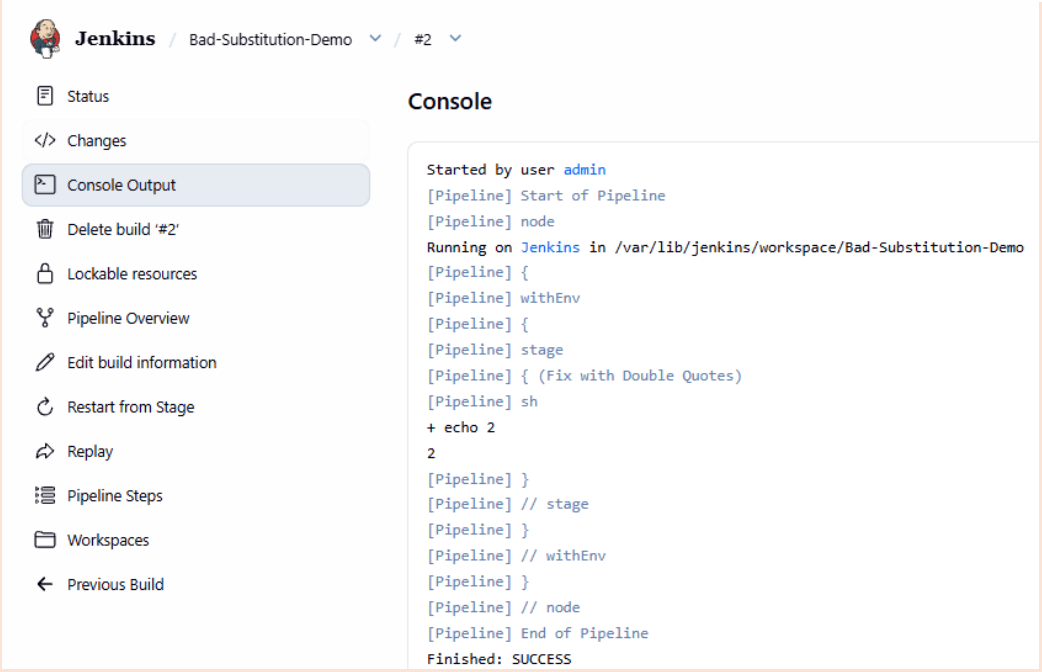
11. Best Practices

Practice	Why
Use double or triple quotes for variable interpolation	Enables substitution
Force #!/bin/bash for complex scripts	Better syntax support
Use withEnv for Jenkins vars that shell must see	Ensures export
Add -xe for debugging	Prints commands and stops on first error
Build incrementally — test each piece	Easier to isolate failures
Re-enable Groovy sandbox after testing	Security
Prefer triple quotes for multi-line commands	Cleaner, no escaping

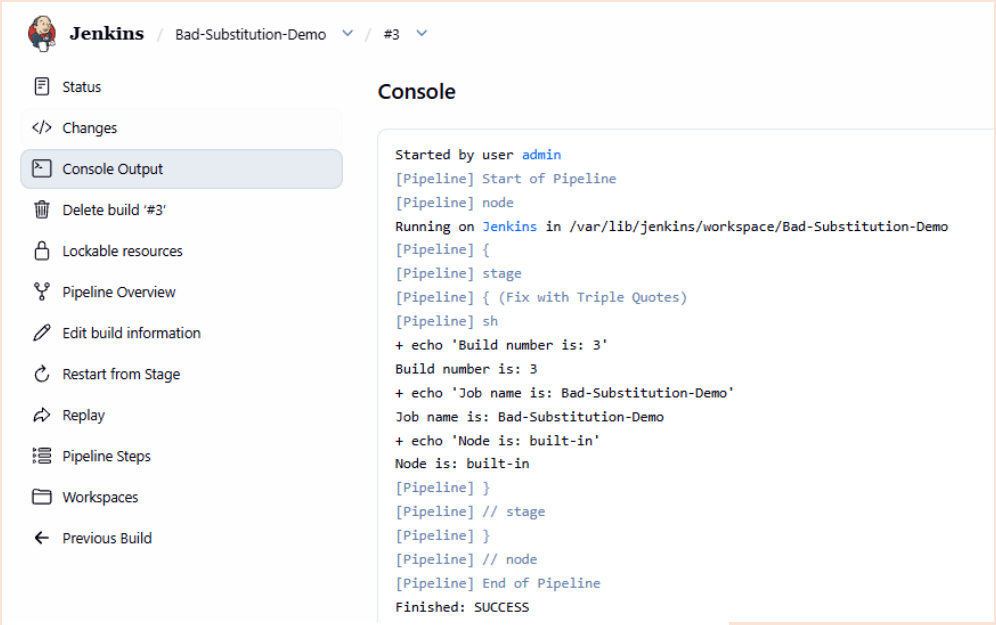
Article-1

Implementation:





Article-1




```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Bad-Substitution-Demo
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Fix with Bash Shebang)
[Pipeline] sh
+ echo 'Running in bash'
Running in bash
+ echo 'Build dir is: /var/lib/jenkins/workspace/Bad-Substitution-Demo/'
Build dir is: /var/lib/jenkins/workspace/Bad-Substitution-Demo/
+ echo 'Build number is: 4'
Build number is: 4
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Conclusion:

The "Bad substitution" error occurs when single quotes prevent Groovy from substituting variables before the shell executes them.

Eliminate this error using five different fixes:

- **Double quotes**: Simple, single-line variable replacement.
- **Triple double quotes**: Multi-line commands with internal quoting.
- **Bash shebang (#!/bin/bash)**: Forces Bash execution with debug capabilities.
- **withEnv block**: Securely exports variables directly into the environment.
- **Combined pattern**: Merges all techniques for a robust, production-ready solution.

Best Practice: Understand Groovy's quoting rules and the shell's substitution mechanism to ensure reliable pipelines when running native OS commands.

Article-1

Task 9: Utilizing Global Variables in Jenkins Pipeline

1. Objective

To understand, access, and utilize Jenkins built-in global variables (env, currentBuild, params) inside pipeline scripts, and to create custom global variables via the Jenkins UI and Groovy scripts.

2. Purpose

Jenkins provides predefined objects and values called **global variables** that are accessible throughout the entire pipeline script without any setup. These variables expose critical information about the build environment, the current build's status, and user-supplied parameters. They are essential for:

- **Making pipelines dynamic** — no more hardcoding job names, build numbers, or paths
- **Conditional execution** — run stages only on specific branches or parameter values
- **Post-build notifications** — send emails with build results and URLs
- **Reusability** — same pipeline script works across jobs and environments
- **Custom org-wide constants** — define variables once, use everywhere
- **Reusable functions** — create custom steps via Groovy scripts in the vars/ folder

Without global variables, every pipeline would need hardcoded values, making it fragile and non-reusable. This task demonstrates both built-in and custom global variables

4. Plugins Needed

Feature	Plugin Required
env variables	None — built into Jenkins core
currentBuild object	None — built into Jenkins core
params object	None — built into Jenkins core
Environment variables (UI config)	None — built into Jenkins core
Global Shared Library (vars/ folder)	None — built into Jenkins core
mail step (for notification example)	**Mailer Plugin** (usually pre-installed)

5. Concepts Reference

5.1 Three Main Types of Global Variables

Variable Type	Purpose	Examples
env	Build environment information	env.BUILD_NUMBER, env.JOB_NAME, env.WORKSPACE, env.BUILD_URL, env.BRANCH_NAME
currentBuild	Information about the current running build	currentBuild.result, currentBuild.duration, currentBuild.previousBuild, currentBuild.fullDisplayName
params	User-supplied build parameters	params.DEPLOY_ENV, params.VERSION

5.2 Access Syntax

Syntax	Meaning
env.VARIABLE_NAME	Full form — always works
${VARIABLE_NAME}	Short form for env.VARIABLE_NAME inside strings
currentBuild.property	Dot notation for currentBuild object
params.PARAM_NAME	Dot notation for parameters

6. Step-by-Step Procedures

1. Click **New Item** → Name: Global-Variables-Demo → Select **Pipeline** → **OK**

Method 1: Using env Variables

```
pipeline {
   agent any
   stages {
      stage('Environment Info') {
         steps {
            echo "Building ${env.JOB_NAME} #${env.BUILD_NUMBER}"
            echo "Workspace: ${env.WORKSPACE}"
            echo "Build URL: ${env.BUILD_URL}"
            echo "Node: ${env.NODE_NAME}"
            echo "Short form: ${BUILD_NUMBER}"
         }
      }
   }
}
```

Breakdown:

Variable	Returns
env.JOB_NAME	The job's name
env.BUILD_NUMBER	The current build number
env.WORKSPACE	Absolute path to the job's workspace
env.BUILD_URL	Full URL of the current build
env.NODE_NAME	Name of the node running the build
${BUILD_NUMBER}	Short form — same as env.BUILD_NUMBER

Method 2: Using currentBuild Variables

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        echo "Starting build..."
        sleep 2
        echo "Build step done."
      }
    }
  }
  post {
    always {
      echo "Build result: ${currentBuild.result}"
      echo "Build duration: ${currentBuild.duration}ms"
      echo "Full display name: ${currentBuild.fullDisplayName}"
      echo "Current build number: ${currentBuild.number}"
    }
  }
}
```

Breakdown:

Variable	Returns
currentBuild.result	SUCCESS, FAILURE, UNSTABLE, or null (if still running)
currentBuild.duration	Duration in milliseconds
currentBuild.fullDisplayName	Full job name + build number
currentBuild.number	Build number (same as env.BUILD_NUMBER)

Note: currentBuild.result is null during the build and only set after the build completes. That's why we read it in the post block.

Method 3: Using params Variables

```
pipeline {
  agent any
  parameters {
    choice(
      name: 'DEPLOY_ENV',
      choices: ['dev', 'staging', 'prod'],
      description: 'Select deployment environment'
    )
    string(
      name: 'VERSION',
      defaultValue: '1.0.0',
      description: 'Version to deploy'
    )
  }
```

```
    stages {
        stage('Deploy') {
            steps {
                echo "Deploying version ${params.VERSION} to ${params.DEPLOY_ENV}"
                echo "Deploy environment: ${params.DEPLOY_ENV}"
                echo "Deploy version: ${params.VERSION}"
            }
        }
    }
}
```

Step 3.3: The first time you click **Build Now**, Jenkins will show a **Build with Parameters** page instead of running immediately.

1. Select DEPLOY_ENV = staging
2. Leave VERSION = 1.0.0 (or change it)
3. Click **Build**

important: Parameters are only available AFTER at least one build has run once. The first Build Now might show parameters as empty if the job was just created. If so, run the job once, then subsequent builds will offer the parameter form.

Method 4: Conditional Execution Based on Branch

```
pipeline {
    agent any
    stages {
        stage('Always Runs') {
            steps {
                echo "This stage always runs."
                echo "Branch name: ${env.BRANCH_NAME}"
            }
        }
        stage('Deploy Only on Main') {
            when {
                expression { env.BRANCH_NAME == 'main' }
            }
            steps {
                echo "🚀 Deploying main branch to production!"
            }
        }
        stage('Deploy Only on Develop') {
            when {
                expression { env.BRANCH_NAME == 'develop' }
            }
            steps {
                echo "🚀 Deploying develop branch to staging!"
            }
        }
    }
}
```

Why BRANCH_NAME **is null:** This job isn't tied to a Git repository, so BRANCH_NAME is not set. In a real multi-branch pipeline (from a Git repo), it would be main, develop, etc.

Note: To test with BRANCH_NAME, use a **Multibranch Pipeline** job connected to a Git repo, or set env.BRANCH_NAME manually

Method 5: Email Notification on Failure (Using currentBuild)

```
pipeline {
   agent any
   stages {
      stage('Build') {
         steps {
            echo "Building..."
         }
      }
   }
   post {
      failure {
         echo "Build failed!"
         echo "Full name: ${currentBuild.fullDisplayName}"
         echo "Build URL: ${env.BUILD_URL}"
         // Uncomment the mail step below only if email is configured
         // mail to: 'team@example.com',
         //      subject: "Failed Pipeline: ${currentBuild.fullDisplayName}",
         //      body: "Something is wrong with ${env.BUILD_URL}"
      }
   }
}
```

Result: The failure block doesn't run because the build succeeds. To test it, add error "forced failure" inside the Build stage.

Method 6: Custom Global Variable via Jenkins UI

Step 6.1: Configure the Environment Variable

1. Go to **Manage Jenkins → System** (or **Configure System**)
2. Scroll to the **Global properties** section
3. Check the box **Environment variables**
4. Click **Add**
5. Enter:
    - **Name:** COMPANY_NAME
    - **Value:** TechCorp
6. Click **Save**

Step 6.2: Use It in a Pipeline

```
pipeline {
   agent any
   stages {
      stage('Welcome') {
         steps {
            echo "Welcome to ${env.COMPANY_NAME}'s Jenkins Pipeline!"
            echo "Company: ${COMPANY_NAME}"
         }
      }
   }
}
```

Note: The variable is now available in **every** pipeline across Jenkins (that runs after this config change)

Method 7: Custom Global Function via Groovy Script (Shared Library)
The final Pipeline used was:

```
@Library('my-shared-library') _

pipeline {
   agent any

   stages {
      stage('Use Custom Function') {
         steps {
            script {
               def timestamp = getTimestamp()
               echo "Current timestamp: ${timestamp}"
            }
         }
      }
   }
}
```

Yes. These are **Jenkins Shared Library-style global functions**. I'll walk you through both examples from the EC2 terminal, then show how to test them in a Pipeline.

Part 1 — Create getTimestamp()
Step 1: Find your Jenkins home
Run:
echo $JENKINS_HOME
If it returns nothing, check:
ls -ld /var/lib/jenkins
Step 2: Create the vars **directory**
sudo mkdir -p /var/lib/jenkins/vars

Step 3: Create getTimestamp.groovy
sudo nano /var/lib/jenkins/vars/getTimestamp.groovy

```
def call() {
   return new Date().format("yyyyMMdd_HHmmss")
}
```

Then verify:
sudo cat /var/lib/jenkins/vars/getTimestamp.groovy

Step 4: Make sure Jenkins can read it
sudo chown -R jenkins:jenkins /var/lib/jenkins/vars
sudo chmod -R 755 /var/lib/jenkins/vars

Step 5: Reload Jenkins configuration

sudo systemctl restart Jenkins
Then check:
sudo systemctl status Jenkins

(or)
Go to **Manage Jenkins → Reload Configuration from Disk**.

it job build is failed then
Create the file
sudo nano /var/lib/jenkins/vars/getLatestTag.groovy

```
def call() {
   return sh(
      script: 'git describe --tags --abbrev=0 2>/dev/null || echo "no-tag"',
      returnStdout: true
   ).trim()
}
```

echo $JENKINS_HOME
Check the directory: sudo ls -ld /var/lib/jenkins

Step 6 – Create a Dedicated Shared Library Directory
A proper Shared Library repository was created at: /opt/jenkins-shared-library
Create the directory: sudo mkdir -p /opt/jenkins-shared-library/vars
Give the current user ownership: sudo chown -R ec2-user:ec2-user /opt/jenkins-shared-library
Move/copy the function into the Shared Library: cp /var/lib/jenkins/vars/getTimestamp.groovy \
/opt/jenkins-shared-library/vars/getTimestamp.groovy

Step 7 – Initialize Git
The Shared Library needs to be stored in Git so Jenkins can retrieve it.
Go to the library directory: cd /opt/jenkins-shared-library
Initialize Git: git init
Set the main branch: git branch -M main

Step 8 – Configure Git Identity
Configure the Git username: git config user.name "Sunith3210"
Configure the Git email: git config user.email "sunithmalem@gmail.com"
Verify if needed: git config --list

Step 9 – Add the Shared Library File
Run: git add vars/getTimestamp.groovy
Check Git status: git status
If the file was already committed, Git may show:
nothing to commit, working tree clean
git commit -m "Add getTimestamp shared library function"

Step 10 – Create GitHub Repository
A GitHub repository was created for the Jenkins Shared Library.
Repository name: jenkins-shared-library
The repository structure should be:
jenkins-shared-library/

└── vars/
  └── getTimestamp.groovy

Step 11 – Connect Local Repository to GitHub
Add the GitHub repository as the remote:
git remote add origin https://github.com/Sunith3210/jenkins-shared-library.git
Verify: git remote -v
Expected:
origin https://github.com/Sunith3210/jenkins-shared-library.git (fetch)
origin https://github.com/Sunith3210/jenkins-shared-library.git (push)
Push the main branch: git push -u origin main

Step 12 – Configure Jenkins Global Shared Library
Open Jenkins: http://localhost:8080
Go to: Manage Jenkins → System
Find:
Global Trusted Pipeline Libraries
Add a new library.
Configure:
Name: my-shared-library
Set:
Default version: main
For the retrieval method, select: Modern SCM
Select:
Git
Enter the GitHub repository: https://github.com/Sunith3210/jenkins-shared-library.git
Save the Jenkins configuration.

Step 13 – Import the Shared Library in the Pipeline
At the beginning of the Jenkinsfile, add:
@Library('my-shared-library') _
This tells Jenkins to load the Shared Library named:
my-shared-library
from the configured Git repository.

8. Verification
Run each method and confirm:

- **Method 1:** Console shows correct job name, build number, workspace path
- **Method 2:** Post-build shows SUCCESS and duration in milliseconds
- **Method 3:** Build with Parameters page appears; selected values are echoed
- **Method 4:** Conditional stages skipped or executed based on branch
- **Method 5:** Failure block does not run on successful builds
- **Method 6:** Custom variable from UI is accessible via env.COMPANY_NAME
- **Method 7:** getTimestamp() returns a valid timestamp string
- **Method 8:** getLatestTag() returns a tag or fallback value

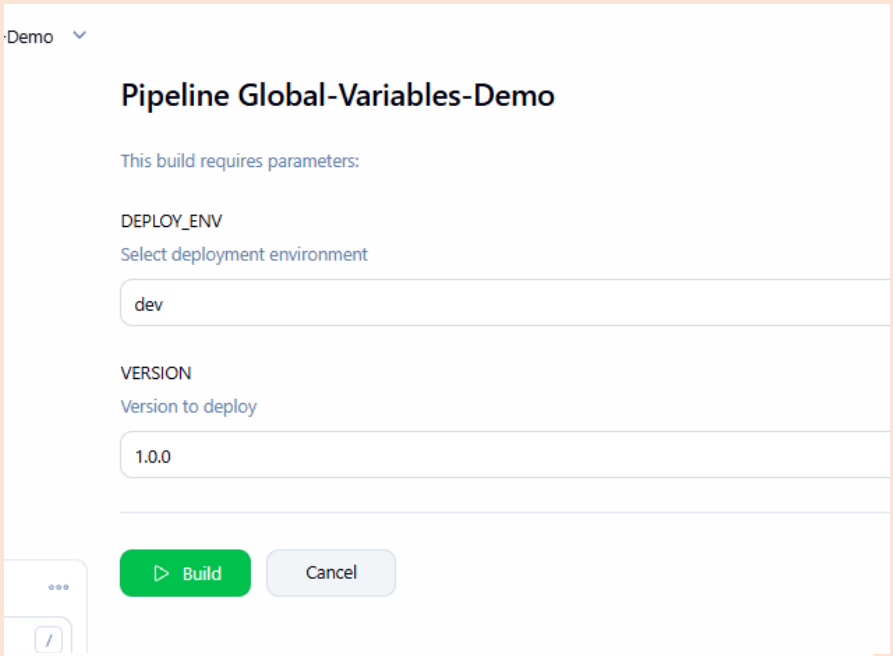
Article-1

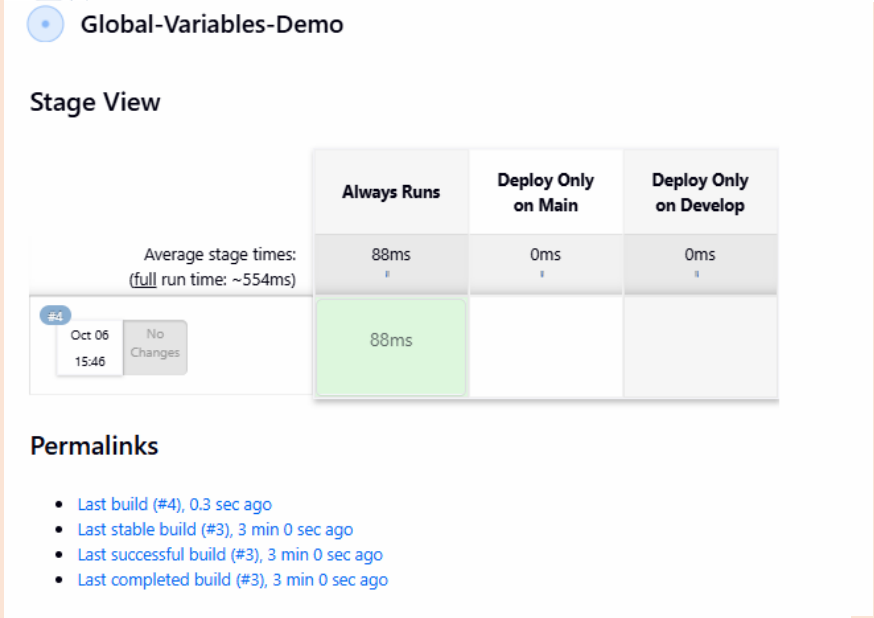
Implementation:

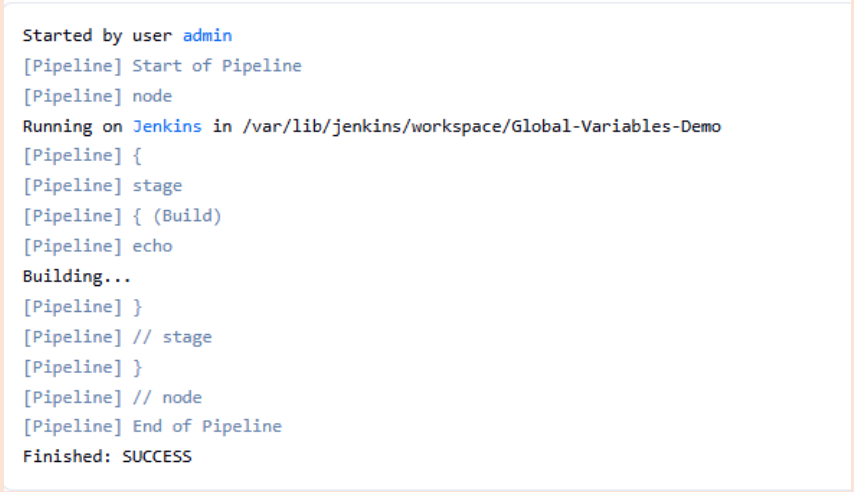
Console

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Global-Variables-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Environment Info)
[Pipeline] echo
Building Global-Variables-Demo #1
[Pipeline] echo
Workspace: /var/lib/jenkins/workspace/Global-Variables-Demo
[Pipeline] echo
Build URL: http://3.90.210.93:8080/job/Global-Variables-Demo/1/
[Pipeline] echo
Node: built-in
[Pipeline] echo
Short form: 1
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

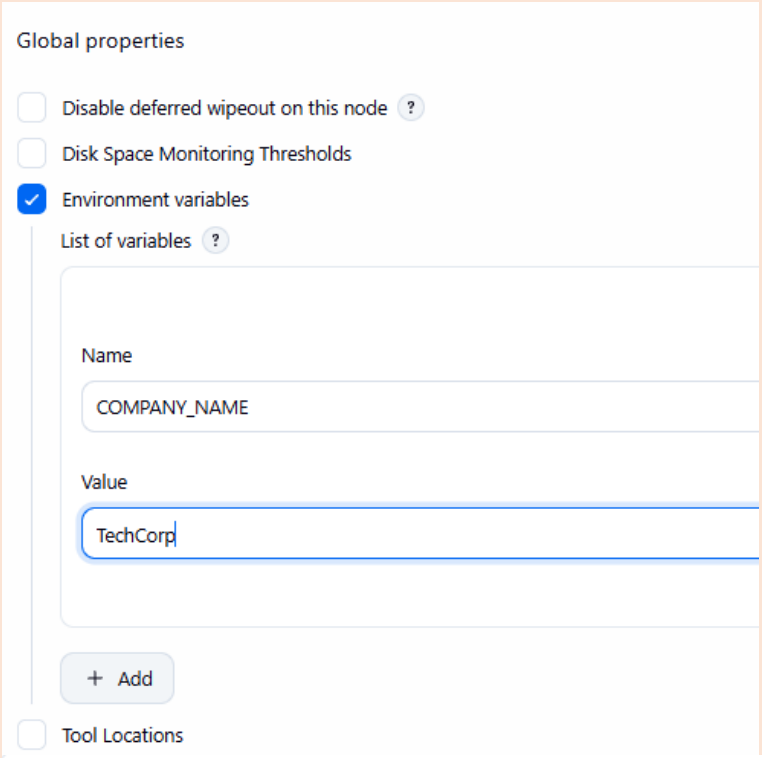

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node (hide)
Running on Jenkins in /var/lib/jenkins/workspace/Global-Variables-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Environment Info)
[Pipeline] echo
Building Global-Variables-Demo #1
[Pipeline] echo
Workspace: /var/lib/jenkins/workspace/Global-Variables-Demo
[Pipeline] echo
Build URL: http://3.90.210.93:8080/job/Global-Variables-Demo/1/
[Pipeline] echo
Node: built-in
[Pipeline] echo
Short form: 1
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```


Demo
Pipeline Global-Variables-Demo
This build requires parameters:
DEPLOY_ENV
Select deployment environment
dev
VERSION
Version to deploy
1.0.0
Build
Cancel

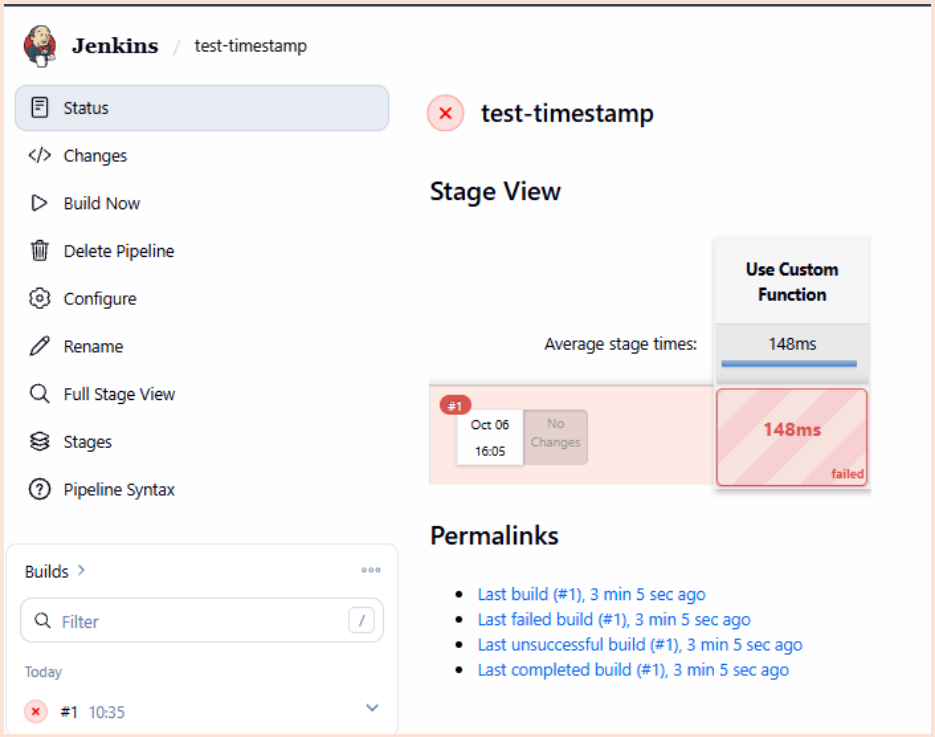

Global-Variables-Demo
Stage View
Always Runs
Deploy Only on Main
Deploy Only on Develop
Average stage times:
(full run time: ~554ms)
88ms
0ms
0ms
#4
Oct 06
15:46
No Changes
88ms
Permalinks
Last build (#4), 0.3 sec ago
Last stable build (#3), 3 min 0 sec ago
Last successful build (#3), 3 min 0 sec ago
Last completed build (#3), 3 min 0 sec ago


Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Global-Variables-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

Article-1




```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Global-Variables-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Welcome)
[Pipeline] echo
Welcome to TechCorp's Jenkins Pipeline!
[Pipeline] echo
Company: TechCorp
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Conlcusion:

Jenkins built-in global variables (env, currentBuild, and params) make pipelines dynamic and reusable by exposing environment details, build statuses, and user inputs.
You can also create custom global variables for advanced configurations:

- **Jenkins UI Globals**: Best for setting organization-wide constants.
- **Groovy Scripts (vars/ folder)**: Best for defining reusable functions across pipelines.

Task 10: Get Last Successful Build Status in Jenkins

1. Objective

To retrieve the last successful build number (and other build statuses) of a Jenkins job using two methods: the Jenkins REST API via curl and the Groovy Jenkins.instance API inside a pipeline.

2. Purpose

Knowing the last build status of a job is critical for intelligent pipeline decisions. Common real-world scenarios:

- Conditional deployment — deploy only if the last build succeeded
- Dashboard reporting — display build statuses across multiple jobs
- Dependency management — trigger downstream jobs only when upstream passed
- Notifications — include last success/failure info in Slack or email alerts
- Rollback decisions — identify the last known-good build for rollback

Article-1

Method 1: Using curl (Command Line — REST API)
Step 1.1: Generate an API Token

1. Log in to Jenkins
2. Click your username (top right corner)
3. Click Configure (left sidebar)
4. Scroll to the API Token section
5. Click Add new Token
6. Give it a name: cli-token
7. Click Generate
8. Copy the token immediately — you won't see it again
9. Click Save

Store the token safely — you'll use it in the next steps.

Step 1.2: Run curl to Get Last Successful Build
Open a terminal on the Jenkins host (or any machine that can reach Jenkins):

curl --user admin:YOUR_API_TOKEN http://localhost:8080/job/File-Exists-Demo-2/lastSuccessfulBuild/buildNumber

Expected output: 4

(Your build number may differ based on your build history.)
Step 1.3: Query Other Build Statuses

```
# Last build (any result)
curl --user admin:YOUR_API_TOKEN http://localhost:8080/job/File-Exists-Demo-2/lastBuild/buildNumber
# Output: 5

# Last failed build
curl --user admin:YOUR_API_TOKEN http://localhost:8080/job/File-Exists-Demo-2/lastFailedBuild/buildNumber
# Output: 5
```

Step 1.4: Get Full JSON Info (Optional)

curl --user admin:YOUR_API_TOKEN http://localhost:8080/job/File-Exists-Demo-2/lastSuccessfulBuild/api/json

Output: A large JSON object with build details — number, result, timestamp, duration, URL, etc.

Method 2: Using Groovy Inside a Jenkins Pipeline
Step 2.1: Create a New Pipeline Job

1. Click **New Item** → Name: Build-Status-Query → Select **Pipeline** → **OK**

Add the Pipeline Script

```
pipeline {
  agent any

  stages {
    stage('Get Build Status') {
      steps {
```

```
            script {
                def jobName = "File-Exists-Demo-2"
                def job = Jenkins.instance.getItem(jobName)

                if (job == null) {
                    echo "Job '${jobName}' not found."
                } else {

                    echo "========================================="
                    echo "Job: ${jobName}"
                    echo "========================================="

                    def lastSuccess = job.getLastSuccessfulBuild()

                    if (lastSuccess != null) {
                        echo "Last successful build: #${lastSuccess.number}"
                        echo "Timestamp: ${lastSuccess.getTimestampString()}"
                        echo "Duration: ${lastSuccess.duration}ms"
                        echo "URL: ${lastSuccess.url}"
                    } else {
                        echo "No successful builds yet."
                    }

                    def lastFailed = job.getLastFailedBuild()

                    if (lastFailed != null) {
                        echo "Last failed build: #${lastFailed.number}"
                    } else {
                        echo "No failed builds."
                    }

                    def lastBuild = job.getLastBuild()

                    if (lastBuild != null) {
                        echo "Last build: #${lastBuild.number}"
                        echo "Result: ${lastBuild.result}"
                    }

                    echo "========================================="
                }
            }
        }
    }
}
```

Script Security Error
When you initially ran the Pipeline, Jenkins may display:
Scripts not permitted to use staticMethod
jenkins.model.Jenkins getInstance

Open Script Approval
Go to: **Manage Jenkins → In-process Script Approval**
Script-Level Approval
In some Jenkins configurations, the first approval can lead to another message:
UnapprovedUsageException: script not yet approved for use

Again go to: **Manage Jenkins → In-process Script Approval**
Look under: **Pending Approvals**

Run the Pipeline Again
Go to: **Build-Status-Query → Build Now**
Open the new build. Select: **Console Output**
The Pipeline should now execute successfully

```
[Pipeline] echo
Last successful build: #4
[Pipeline] echo
Timestamp: 7 hr 1 min
[Pipeline] echo
Duration: 1429ms
[Pipeline] echo
URL: job/File-Exists-Demo-2/4/
[Pipeline] echo
Last failed build: #5
[Pipeline] echo
Last build: #5
[Pipeline] echo
Result: FAILURE
[Pipeline] echo
==========================================
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Method 3: Conditional Deployment Based on Last Build Status
Step 3.1: Create a Deployment Decision Pipeline

1. Click **New Item** → Name: Deploy-If-Last-Success → Select **Pipeline** → **OK**

```
pipeline {
  agent any

  stages {

    stage('Check Upstream Build') {
      steps {
        script {
```

```
                def upstreamJob = "File-Exists-Demo-2"

                def job = Jenkins.instance.getItem(upstreamJob)

                if (job == null) {
                    error "Upstream job '${upstreamJob}' does not exist."
                }

                def lastBuild = job.getLastBuild()

                if (lastBuild == null) {
                    error "No builds found for '${upstreamJob}'."
                }

                if (lastBuild.result == 'SUCCESS') {

                    echo "Last build of '${upstreamJob}' succeeded (#${lastBuild.number}). Proceeding."

                } else {

                    error "Last build of '${upstreamJob}' was ${lastBuild.result}. Deployment stopped."
                }
            }
        }
    }

    stage('Deploy') {
        steps {

            echo "Deploying..."

            echo "Deployment complete."
        }
    }
  }
}
```

In this task:

- File-Exists-Demo-2 is the **upstream job**.
- Deploy-If-Last-Success is the **downstream/deployment job**.
- If the latest upstream build is **SUCCESS**, deployment continues.
- If the latest upstream build is **FAILURE**, deployment stops.

Possible results include:

SUCCESS
FAILURE
ABORTED
UNSTABLE

Article-1

Script Security Approval
For example:
Scripts not permitted to use staticMethod
jenkins.model.Jenkins getInstance

Later:
Scripts not permitted to use method
hudson.model.ItemGroup getItem java.lang.String

And:
Scripts not permitted to use method
hudson.model.Job getLastBuild
This happens because Jenkins protects Pipeline scripts from accessing sensitive internal APIs.

Approving the Required Signatures
Go to: **Manage Jenkins → In-process Script Approval**

What is API in this
Your pipeline:
Pipeline → Jenkins internal API → Job object → Build status

The task demonstrates two approaches: Jenkins REST API using curl, and Jenkins internal API using Groovy. The Deploy-If-Last-Success pipeline specifically uses the internal Jenkins API, not a REST API cal

API = a way for software to ask Jenkins questions and receive Jenkins's answer

→ yes — the main purpose of this task is checking another Jenkins job's build status and using that information to make a pipeline decision

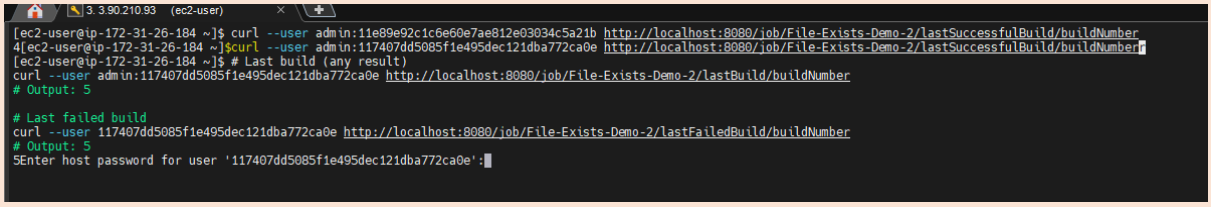
Method 1 = curl **+ REST API**
Method 2 = Groovy + Jenkins internal API
Practical example = Don't deploy if the previous/latest job failed

Article-1

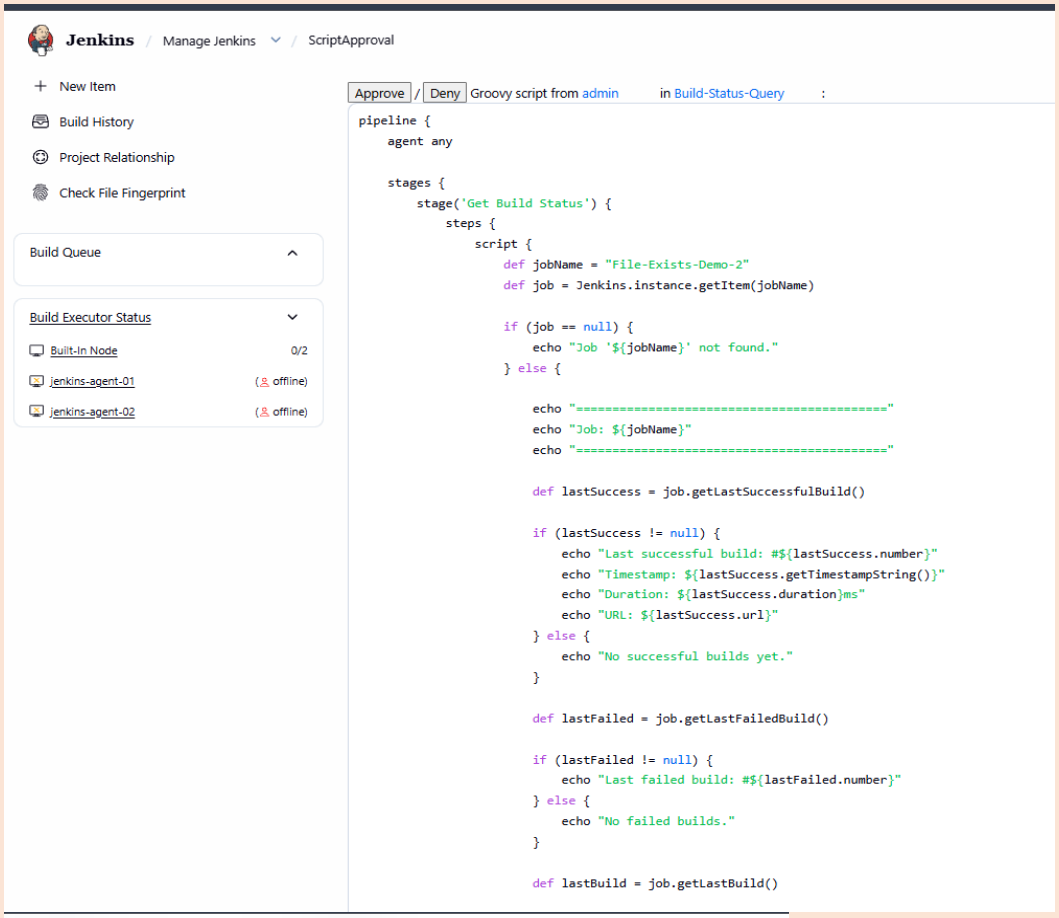
Best Practices

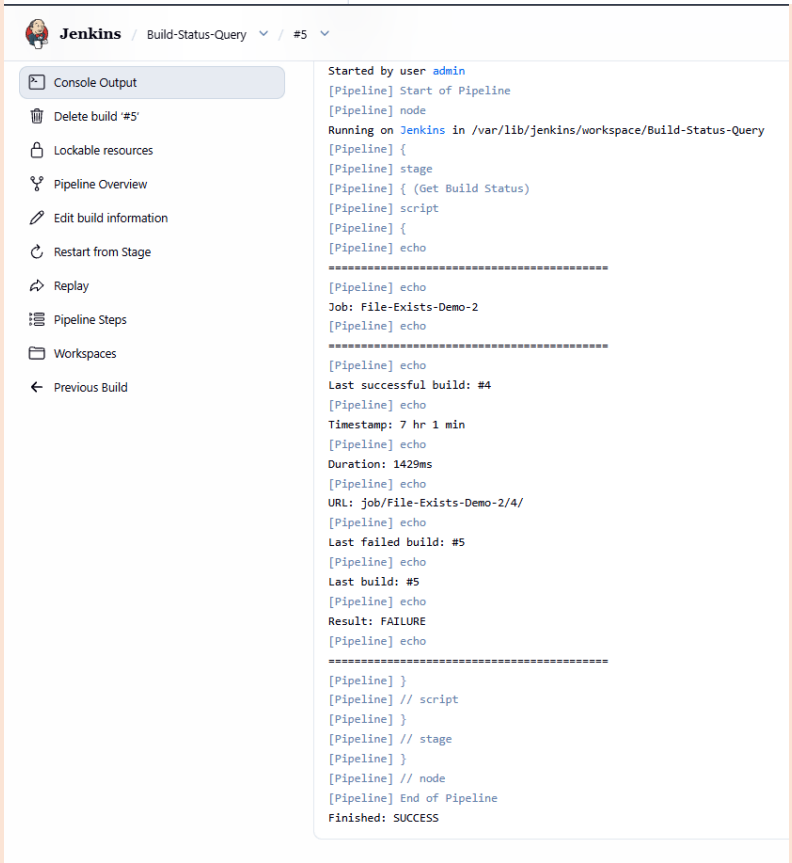
Practice	Why
Always use API tokens, never passwords	Tokens can be revoked independently
Store tokens in Jenkins credentials, not scripts	Avoids exposure in logs
Use Jenkins.instance for internal queries	No token needed, cleaner
Check for null before accessing build properties	Prevents NullPointerException
Use lastSuccessfulBuild for deployment gates	Most reliable "known good" indicator
Log build numbers in console for traceability	Easier debugging
Prefer lastStableBuild over lastSuccessfulBuild for rollback	Stable = passed all tests, successful = passed build

Implementation :



Article-1


Jenkins / Manage Jenkins / ScriptApproval
New Item
Build History
Project Relationship
Check File Fingerprint
Build Queue
Build Executor Status
Built-In Node 0/2
jenkins-agent-01 (offline)
jenkins-agent-02 (offline)
Approve / Deny Groovy script from admin in Build-Status-Query :
pipeline {
agent any
stages {
stage('Get Build Status') {
steps {
script {
def jobName = "File-Exists-Demo-2"
def job = Jenkins.instance.getItem(jobName)
if (job == null) {
echo "Job '${jobName}' not found."
} else {
echo "=="
echo "Job: ${jobName}"
echo "=="
def lastSuccess = job.getLastSuccessfulBuild()
if (lastSuccess != null) {
echo "Last successful build: #${lastSuccess.number}"
echo "Timestamp: ${lastSuccess.getTimestampString()}"
echo "Duration: ${lastSuccess.duration}ms"
echo "URL: ${lastSuccess.url}"
} else {
echo "No successful builds yet."
}
def lastFailed = job.getLastFailedBuild()
if (lastFailed != null) {
echo "Last failed build: #${lastFailed.number}"
} else {
echo "No failed builds."
}
def lastBuild = job.getLastBuild()


Jenkins / Build-Status-Query / #5
Console Output
Delete build '#5'
Lockable resources
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Build-Status-Query
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Get Build Status)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
==========
[Pipeline] echo
Job: File-Exists-Demo-2
[Pipeline] echo
==========
[Pipeline] echo
Last successful build: #4
[Pipeline] echo
Timestamp: 7 hr 1 min
[Pipeline] echo
Duration: 1429ms
[Pipeline] echo
URL: job/File-Exists-Demo-2/4/
[Pipeline] echo
Last failed build: #5
[Pipeline] echo
Last build: #5
[Pipeline] echo
Result: FAILURE
[Pipeline] echo
==========
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

Article-1

Conclusion:
Jenkins core provides two built-in ways to retrieve a job's last successful build status without requiring additional plugins.
Choose the method that fits your access location:

- **REST API via curl**: External and token-based; ideal for external scripts, tools, and dashboards.
- **Groovy Jenkins.instance API:** Internal and token-free; ideal for direct use inside pipelines.